

QST Exploring RF—1995

This file contains the six Exploring RF columns from the 1995 issues of *QST*. Users of *ARRL Radio Designer* software will find these columns to be useful application notes that explore ways of using *ARRL Radio Designer* to solve thorny circuit-analysis problems of the sort encountered in RF design.

If you are using Adobe *Acrobat* version 2.1, with its WEBLINK.API plug-in, and if you have a World Wide Web browser installed and configured to work with *Acrobat*, you can click on the links in these articles to download referenced files from the ARRL Web server or to generate an email message to a referenced email address.

Enjoy. And let these be only your starting point for *using ARRL Radio Designer* to help you design and analyze the circuits in your next RF project!

Now that *ARRL Radio Designer* is on the street, we're hoping that ham interest in exploring radio-frequency (RF) technology through computer-based modeling will take off in a way that hasn't been possible before. This column is part of taking that show on the road—a place for suggesting *ARRL Radio Designer* applications we hope will be interesting to many hams.

But this column won't stop there. Many useful electronic and RF circuit design programs have been with us for years in various forms, and new ones keep appearing. For starters, I'm thinking of the various commercial products based on the University of California at Berkeley's *SPICE* (Simulation Program with Integrated Circuit Emphasis)—at least one of which, MicroSim Corp's *PSpice*, is available free in a limited-function evaluation version, and other of which, *MICRO-CAP*, is available jointly from Addison-Wesley Publishing and Benjamin/Cummings Publishing in a low-cost student edition. Just sketching the abilities of the many *SPICE* derivatives would take several pages. A diskful of antenna, transmission-line and propagation software now ships with every new *ARRL Antenna Book* sold, and software is also available with *The ARRL UHF/Microwave Experimenter's Manual*—what can those programs do? And what about the freeware and shareware that's almost everywhere? There's a lot to be excited about if you're interested in using computer-aided design (CAD) and computer-aided engineering (CAE) in Amateur Radio, and *QST* will share that excitement through Exploring RF.

Optimizing Circuit Performance with *ARRL Radio Designer*

One of the *ARRL Radio Designer* features we're most excited about is the program's ability to *optimize* circuits—to make circuits work better, or to modify circuit performance according to specified goals. I suggested in October *QST*'s *ARRL Radio Designer* article¹ that one use for this feature might be to tweak a 2-meter receiver preamp for a better noise figure, but that's a pretty far-out example for many of us. Then I went on to describe an antenna-tuner optimization that's a very nice example of optimizing a circuit almost, but not exactly, like the one actually shown.²

Elsewhere in this issue, Andy Griffith's "Getting the Most Out of Your T-Network Antenna Tuner" includes some eye-opening findings on just how lousy a T network can be when transforming some impedances to 50 Ω . *ARRL Radio Designer*, manipulated by yours truly, backed up Andy's findings and even provided a few graphics. Here's how I did it—and how you can use *ARRL Radio Designer*'s optimizer to simulate adjustable T networks, too.

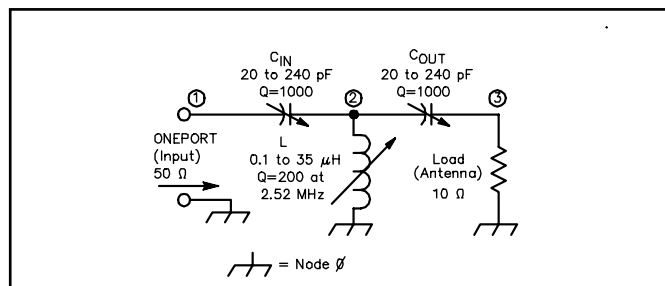


Figure 1—Many of us use a capacitor-inductor-capacitor T network similar to this one as an antenna tuner. With an SWR or reflected power meter acting as an indicator, we adjust its C and L values to transform the antenna's impedance (here, 10 Ω , resistive) to 50 Ω , resistive (50+ $j0$ Ω when written in rectangular notation) at our frequencies of interest. (I specify L's Q at 2.52 MHz to tie in with lab Q-meter conventions, as pages 5-7 and 5-8 of *The ARRL Radio Designer Manual* explain.) The circled numbers indicate nodes for *ARRL Radio Designer* analysis, which treats the tuner as a one-port network imaginatively named ONEPORT.

The T Network in Schematic and Netlist Form

Figure 1 shows Andy's T net's schematic with a few additions and changes. Here's the corresponding *ARD* netlist block:

```
BLK
CAP 1 2 C=?20PF 82.1733PF 1000PF? Q=1000 ; C/IN\
IND 2 0 L=?0.1UH 30.2528UH 35UH? Q1=200 F=2.52MHZ;L
CAP 2 3 C=240PF Q=1000 ; C/OUT\
RES 3 0 R=10 ; this is the load
ONEPORT:1POR 1
END
```

Question marks bracket the component values we want to optimize. Note that instead of just specifying a single value, we've indicated *ranges* for C_{IN} and L. (In keeping with Andy Griffith's suggested tuning technique, we start out by setting C_{OUT} to a fixed value and leaving it alone.) In *ARD*-speak, this *constrains* their values to the ranges shown in Figure 1. Note also that the optimizer won't touch the Q and F values associated with C_{IN} and L. This is appropriate, considering that we can't adjust them at will in a real antenna tuner.

We're interested in seeing how the network plays at 160 meters, so we'll set up a frequency block that combines good resolution across the band with enough wide-range exponential steps for graphing the network's harmonic reduction:

```
FREQ
STEP 1.8MHZ 2.0MHZ 1KHZ
ESTP 0.5MHZ 30MHZ 300
END
```

Finally, we add an optimization (OPT) block that tells *ARD* which circuit to optimize—a netlist may contain more than one, after all—at what frequency or frequencies, in what way, and to what degree:

```
OPT
ONEPORT
F=1.9MHZ
RZ11=50
IZ11=0
TERM=1E-5
END
```

This block tells *ARRL Radio Designer* to optimize the performance of the circuit block ONEPORT so that, at 1.9 MHz, the real (resistive) part of its impedance is 50 Ω and the imaginary (reactive) part of its impedance is $j0$ Ω ; that is, 50+ $j0$ Ω . The TERM=1E-5 line tells *ARD* to terminate optimization—that is, to consider the optimization goals reached—when the optimizer's error function reaches or drops below 1×10^{-5} , or 0.00001. The OPT and END statements merely identify the block and delimit it so *ARRL Radio Designer* knows where it starts and stops. Table 1 shows the corresponding *ARD* netlist, all set to go.

This column isn't the place for excruciating detail about exactly how *ARD* calculates the optimizer's error function. (*The ARRL Radio Designer Manual* explains how optimization goals factor into the error function, and how you can specify their *weight*—the degree to which they contribute to the number.) To get started with optimization, all we need to know is that the greater the error function, the farther we are from a successful optimization.

Running It

Once we've got our optimizable file ready to go, we click on **Optim** (or press Shift+F10). *ARRL Radio Designer* analyzes the circuit first, and then pops its Optimization and Optimization Data dialogs (Figure 2).³

When the optimizer succeeds in pushing its error function to a number equal or less than the OPT block's TERM value—or to 0 if you didn't specify one—it halts, and the message "Optimization Completed" appears in the Optimization dialog.⁴ Click the **Done**

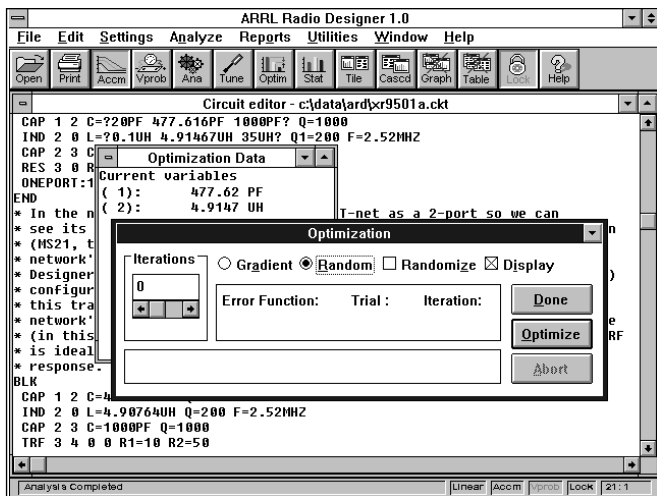


Figure 2—Its pre-optimization analysis complete, *ARRL Radio Designer* pops its Optimization and Optimization Data dialogs. All we need to do now is enter a number in the Iterations edit control (I always enter 1000), clear the **Display** box (*ARD*'s optimizer runs much faster when it doesn't stop to do display calculations between iterations), and click **Optimize**. Leaving the optimizer set to its **Random** default is best for now; we'll have to discuss random versus gradient optimization some other time.

Table 1
Simulating an Adjustable T-Network Antenna Tuner with *ARRL Radio Designer*

```
BLK
CAP 1 2 C=?20PF 82.1733PF 1000PF? Q=1000 ; C/IN\
IND 2 0 L=?0.1UH 30.2528UH 35UH? Q1=200 F=2.52MHZ;L
CAP 2 3 C=240PF Q=1000 ; C/OUT\
RES 3 0 R=10 ; this is the tuner's load
ONEPORT:1POR 1
END
FREQ
STEP 1.8MHZ 2.0MHZ 1KHZ
ESTP 0.5MHZ 30MHZ 300
END
OPT
ONEPORT
F=1.9MHZ
RZ11=50
IZ11=0
TERM=1E-5
END
```

button. "Optimization complete," responds *ARRL Radio Designer*. "Do you want to update circuit?" Click **Yes** (if you don't want to keep and use the updated values, click **No**), and *ARD* updates each optimizable component's initial nominal value with the new value found by the optimizer. A cheerful "WARNING - REANALYSIS IS REQUIRED AFTER OPTIMIZATION" message appears in the screen's lower left corner. When you see this, just click the **Ana** button or press F10.

If we want to determine the tuner's network's frequency response, we need to reconfigure it as a two-port network to see how its amplitude response varies with frequency. Table 2 shows how to do that, and Figure 3 graphs the analysis results based on this approach.

Specifying Reactive Antenna Loads

So far, we've used optimization only in transforming purely resistive loads to $50\ \Omega$ resistive— $50+j0\ \Omega$. In the **ONEPORT** block for our T-net circuit file, we specified the load with the netlist line **RES 3 0 R=10**—a $10+j0\text{-}\Omega$ load. Often, however, the antenna-system load we want to transform to $50+j0\ \Omega$ will be reactive. Its

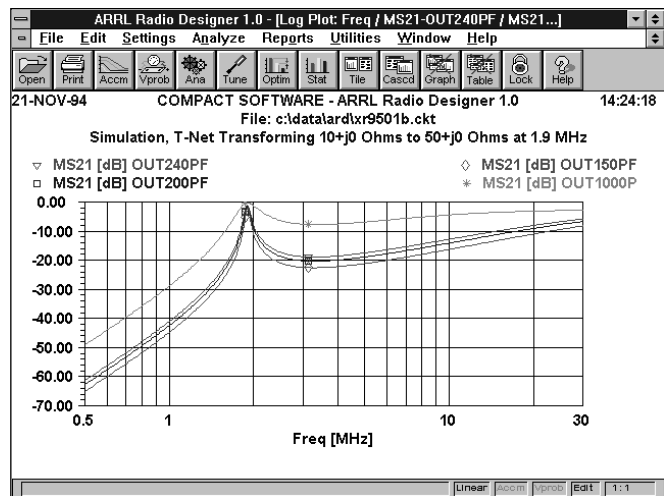


Figure 3—The 160-meter tuner's frequency response for $10+j0$ to $50+j0\ \Omega$ transformations based on various C_{OUT} settings. I used netlist blocks like that shown in Table 3 to do these analyses. If harmonic rejection is what you're after, the capacitor-inductor-capacitor T network isn't the way to get it.

Table 2
The T Network Reconfigured as a Two-Port for Frequency-Response Simulation (Values for a 240-pF C_{OUT} Shown)

```
BLK
CAP 1 2 C=122.811PF Q=1000
IND 2 0 L=19.3847UH Q=200 F=2.52MHZ
CAP 2 3 C=240PF Q=1000
TRF 3 4 0 0 R1=10 R2=50
OUT240PF:2POR 1 4
END
```

This block replaces **ONEPORT**'s **RES** line with an ideal transformer (**TRF**) connected between nodes 3 and 4, with node 0 common for both windings. The phrase **R1=10 R2=50** defines the transformer as presenting $10\ \Omega$ to the network when its secondary is terminated by $50\ \Omega$ —which it will be when we plot **OUT240PF**'s response as the S-parameter **MS₂₁** using *ARD*'s default $50+j0\ \Omega$ terminations. Note: Use this block in addition to, not in place of, the **ONEPORT** block in Table 1.

Table 3
Simulating a T-Network Tuner Feeding a Reactive Antenna at 7.2 MHz

```
BLK
CAP 1 2 C=?20PF 100PF 240PF? Q=1000 ; C/IN\
IND 2 0 L=?0.1UH 30.2528UH 35UH? Q1=200 F=2.52MHZ;L
CAP 2 3 C=240PF Q=1000 ; C/OUT\
ONE 3 0 ANTENNA
ONEPORT:1POR 1
END
FREQ
STEP 7.1MHZ 7.3MHZ 1KHZ
ESTP 1MHZ 30MHZ 300
END
OPT
ONEPORT
F=7.2MHZ
RZ11=50
IZ11=0
TERM=1E-5
END
DATA
ANTENNA: Z RI
60 +60
END
```

j (imaginary) part will be a nonzero positive or negative number. How do we specify that?

If we happen to know what series or parallel combination of resistance and capacitance (or inductance) will exhibit that load at our frequency of interest, we can connect them between nodes 3 and 0 of ONEPORT and optimize away. If we don't, *ARRL Radio Designer's* ONE element, a programmable one-port "black box," can serve.

ONE is a neat solution because we can specify its characteristics in terms of S, Y or Z parameters. Using ONE is a bit more involved than just coding in a few components, however, because we need to specify it *and its impedance data* in the netlist. That means our netlist will have to include a DATA block.

Table 3 puts it all together. The netlist line ONE 3 0 ANTENNA connects a ONE element between nodes 3 and 0 and labels it ANTENNA. The label is important because *ARRL Radio Designer* needs it to link this particular ONE—a more complex netlist might contain several different ONES, after all—with its data in the DATA block:

```
DATA
ANTENNA: Z RI
60 +j60
END
```

This DATA block tells *ARD* to provide impedance (Z) data in real-imaginary (RI) form for the element labeled ANTENNA—in this case, an inductively reactive load of $60+j60\ \Omega$ at all frequencies.⁵ The modeling's yours to do, but here's a hint: The optimizer won't reach its goal with C_{OUT} fixed at 240 pF. You'll have to try a lower value, or make C_{OUT} optimizable.

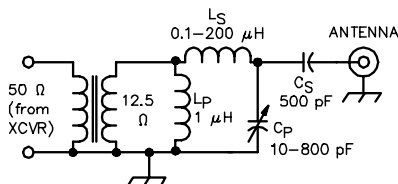
Free Electronic Goodies

You can get expanded versions of the Table 1 and Table 3 netlists, as well as the one I used to generate Figure 3, in the file EXRF9501.TXT, available from the ARRL HQ BBS (203-666-0578) and Internet info server (info@arrl.org). As always, the latest *ARRL Radio Designer* news is also available from those sources in the file ARD.TXT.

Notes

¹David Newkirk, WJ1Z, "Introducing ARRL Radio Designer: New Soft-

ware for RF Circuit Simulation and Analysis," *QST*, Oct 1994, pp 21-26. The "oops" is that I misspecified one of the network's capacitors in the netlist. *ARRL Radio Designer* found a match with C_p , shown in the article's Figure 9 as connected to common on the left side of L_s , actually connected to common from the junction of L_s and C_s . So the network I actually optimized was:



and not *exactly* the tuner described by Rohde. Having reoptimized the circuit after finding and fixing that error, I can now report that the good news is that Rohde's network can indeed transform a $1.6-j2246\ \Omega$ load to $50+j0\ \Omega$ at 1.83 MHz if modified as shown above, but that the bad news is that it *can't* transform that load to $50+j0\ \Omega$ at 1.83 MHz as it was originally specified by Rohde. Live and learn. The corrected circuit file is now part of the information file ARD.TXT, available from the ARRL BBS (203-666-0578) and the ARRL Technical Information Service's Internet-accessible information server (info@arrl.org).

³If you've set *ARD's* Auto Reporter to pop Report Editor or Quick Reporter after every analysis, the one you've selected will pop before the Optimization dialogs appear, and you may even have to close it or move it to work with the Optimization dialogs. Frankly, Auto Reporter irks me—I can't stand that big Linear Reports dialog popping when all I want to do is load predefined reports, which pressing F7 after analysis handily facilitates—so I keep Auto Reporter set to None.

⁴Well, okay—*ARD's* optimizer also stops if it completes the specified number of iterations before finding a solution. The thing to do then—depending on whether you think a solution is possible—is to specify more iterations. If the error function hasn't been dropping through many iterations, a successful solution may not be possible within the value limits set by the netlist.

⁵If we don't specify a frequency for the ONE data, *ARRL Radio Designer* assumes that we mean "at all frequencies"—and this is important. Just as the purely resistive tuner load we've specified so far is unrealistic because no antenna exhibits a constant, pure resistance across even a small frequency range, a reactive load that doesn't vary with frequency is also unrealistic. *ARRL Radio Designer* lets us specify frequencies for black-box-element data, so if we happened to know our antenna's actual impedance across the 1.8 to 2.0-MHz range, we could exactly specify it in the DATA block with multiple frequency-specific lines. Note, though, that it's perfectly okay to use a frequency-nonspecific impedance value in adjusting/optimizing a tuner at any one frequency. **QST**

New Books

SHORTWAVE RADIO LISTENING FOR BEGINNERS and

THE SHORTWAVE LISTENER'S Q & A BOOK

By Anita Louise McCormick, KA8KGI

Published by TAB Books, Division of McGraw-Hill Inc, Blue Ridge Summit, PA 17294-0850; $9\frac{1}{4}\times 7\frac{1}{4}$ inches, black-and-white photos and drawings. *Shortwave Radio Listening for Beginners*, First edition 1993, 176 pp, \$10.95; *The Shortwave Listener's Q & A Book*, First edition 1994, 144 pp, retail price \$12.95.

Reviewed By Brian Battles, WS1O
QST Features Editor

With the publication of these two books, it appears that Anita McCormick is establishing herself as America's "First Lady of Shortwave Listening." She certainly logs an enormous amount of shortwave radio listening (SWLing) time from her home in Huntington, West Virginia, and she's enthusiastic and knowledgeable in her work to bring the hobby into the home of anyone who reads her books.

Shortwave Radio Listening for Beginners is a clear, nontechnical introduction to SWLing for anyone who has yet to venture into the airwaves outside the amateur bands. Anita briefly covers the beginnings of radio and a little bit of broadcasting history, and then goes on to discuss domestic AM broadcast DXing, VHF/UHF scanning, ham radio and more. She lists popular frequencies and modes for worldwide reception of broadcasters, utility, relay, government, pirate and clandestine stations. There are tips on the kinds of receive-

ers, antennas and accessories to consider for your home listening station, and on collecting QSL cards and other mementos and souvenirs from SW broadcasters. Amateurs who operate HF probably already know most of the necessary theory regarding HF antennas, reception and equipment, although you may find a few pointers about the slightly different requirements of setting up an effective SW monitoring station.

It's also gratifying to see her mention the growing use of ham radio in schools and through the US space program, such as shuttle communications. The educational aspects of Amateur Radio are important, especially in a book for listeners, because stumbling across ham communication while tuning for other radio services is often a young person's first exposure to Amateur Radio.

The final chapter covers radio waves, propagation and other such information that may help a listener better understand how and why he can hear different signals at different times, and help determine the best antenna(s) and listening times to hear the stations of interest.

Three adequate appendices list SW stations, radio listening clubs and radio sources. There's a handy glossary and reasonably comprehensive index.

Although it's complete as a standalone work, *The Shortwave Listener's Q & A Book* is a handy companion volume. It depends on whether you prefer a straight text or question-and-answer format. It covers essentially the same topics, but organized in a style that might be more appealing to some readers.

Both books are illustrated with many black-and-white photographs and helpful diagrams, and the writing style is friendly and nontechnical. So if you know a new ham or nonham who has yet to be introduced to listening to shortwave radio and scanners, one or both of these books could make a fine gift...or as a loan from your personal radio library. At just a little more than 10 bucks each, I rate these books a good value for someone starting to get curious about the signals waiting all across the busy RF spectrum. **QST**

Modeling a Direct-Conversion Receiver's Audio Response and Gain with ARRL Radio Designer

You can just about go bonkers in getting started with a powerful tool like *ARRL Radio Designer*¹ if doing so means doing too many all-new things at once. That's one of the reasons why *ARD* ships with seven example circuits:²

- a double-tuned-circuit filter for 13.975 to 14.475 MHz
- a gang-tuned double-tuned-circuit filter for 80 meters
- a simple JFET MF/HF preamp
- a BJT post-mixer amplifier with feedback
- an op-amp audio filter
- a six-crystal ladder filter for 3.579545 MHz
- a 9-MHz diplexer

The examples are there to help you learn the *ARRL Radio Designer* way of doing things, of course, but they are also intended to demonstrate that *ARD* can do a good job of telling you things you already know. (Maybe you don't check each year's new *White Pages* to see if your phone number is correct, but I use that simple test to determine whether or not there's hope for the rest of the book!) Once you're satisfied that a new tool can do the old jobs you know, you're more likely to trust it to do new jobs your old tools can't do.

This month's modeling example—simulating the audio channel of a heterodyne direct-conversion receiver³—is another potential *ARD* trust-builder. Many of us have built heterodyne direct-conversion receivers, so we know what they do: They heterodyne RF signals to AF, amplify the resulting audio, and shove it at us through electroacoustic transducers (headphones or speakers). (Actually, this is also all a superheterodyne receiver ultimately does. The difference is that a superhet also does one or more RF-to-RF conversions before completing that final RF-to-AF conversion.)

Although *ARRL Radio Designer* can't model frequency con-

version or transducer action, it can make itself pretty useful in helping us understand, design, and modify the linear subsystems in a direct-conversion radio. Roger Hayward's "Ugly Weekender" receiver (June 1992 *QST*⁴)—a sound, friendly direct-conversion design for 7 MHz—is a good example for study because most of its active devices are easily modelable discrete transistors. (I personally like the Ugly Weekender because it's easy to build, uses no hard-to-get parts, and sounds like a real radio. Connect it and its companion 1.5-W transmitter to a low half-wave dipole just after lunch some Saturday or Sunday, and you'll have to work hard *not* to contact someone on QRP CW.)

The entire Ugly Weekender Receiver (UWR) schematic covered a full *QST* page. Our *ARD*ized UWR schematic (Figure 1) takes less space because we need to model only four of the original's 11 transistors—just those that amplify and band-limit the UWR's audio. Table 1 shows Figure 1 in *ARRL Radio Designer* netlist form.

Netlist Highlights

Structure

How we structure an *ARRL Radio Designer* netlist for a particular circuit depends somewhat on what we want to know about the circuit we're modeling. We know, for instance, that a direct-conversion receiver containing little or no RF amplification—the UWR uses none—must be capable of something like 80 to 100 dB of audio gain. We also know (by carefully reading citation trails through several articles' worth of footnotes) that the UWR's first audio preamp was designed to terminate the receiver's double-balanced diode mixer in something reasonably close to 50 Ω , resistive, at least in the audio range. Since we're out to see whether *ARRL Radio Designer* can correctly tell us things we already know, what we already know is what we want to find out.

The Table 1 netlist represents the Ugly Weekender Receiver in three circuit blocks—MIXER, Q8-9-10 and Q12—so we can

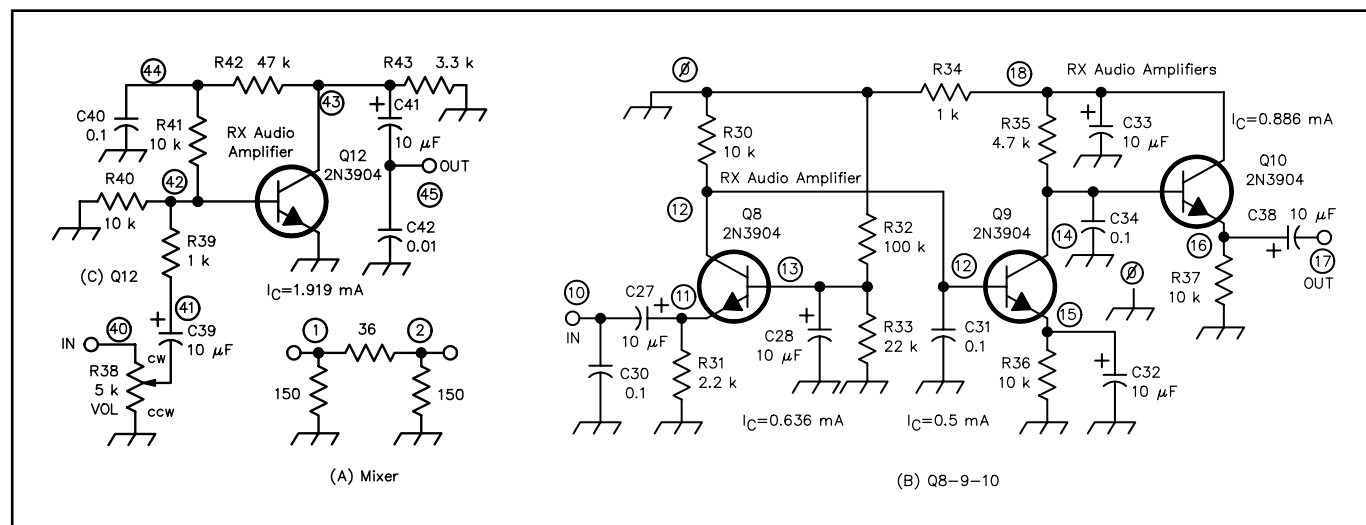


Figure 1—We need concern ourselves with only four of the Ugly Weekender Receiver's 11 transistors in modeling its basic performance with *ARRL Radio Designer*. I added the circled node numbers as I coded the circuit into an *ARD* netlist (Table 1).

Table 1

Simulating the Ugly Weekender Receiver's Audio Amplifier with ARRL Radio Designer

```

BLK ; this is MIXER (Figure 1A)
RES 1 0 R=150
RES 1 2 R=36
RES 2 0 R=150
MIXER:2POR 1 2
END
BLK ; this is Q8-9-10 (Figure 1B)
CAP 10 0 C=0.1UF ; C30
CAP 10 11 C=10UF ; C27
BIP 13 12 11 A=0.99 RE=(26/0.636); Q8
RES 12 0 R=10KOH ; R30
CAP 13 0 C=10UF ; C28
RES 13 0 R=22KOH ; R33
RES 13 0 R=100KOH ; R32
CAP 12 0 C=0.1UF ; C31
BIP 12 14 15 A=0.99 RE=(26/0.5); Q9
RES 15 0 R=10KOH ; R36
CAP 15 0 C=10UF ; C32
RES 14 18 R=4.7KOH ; R35
RES 18 0 R=1KOH ; R34
CAP 18 0 C=10UF ; C33
CAP 14 0 C=0.1UF ; C34
BIP 14 18 16 A=0.99 RE=(26/0.886); Q10
RES 16 0 R=10KOH ; R37
CAP 16 17 C=10UF ; C38
Q8-9-10:2POR 10 17
END
BLK ; this is Q12 (Figure 1C)
RES 40 0 R=5KOH ; R38 (GAIN control at max)
CAP 40 41 C=10UF ; C39
RES 41 42 R=1KOH ; R39
RES 42 0 R=10KOH ; R40
RES 42 44 R=10KOH ; R41
CAP 44 0 C=0.1UF ; C40
RES 44 43 R=47KOH ; R42
BIP 42 43 0 A=0.99 RE=(26/1.919); Q12
RES 43 0 R=3.3KOH ; R43
CAP 43 45 C=10UF ; C41
CAP 45 0 C=0.01UF ; C42
Q12:2POR 40 45
END
BLK ; this block chains MIXER, Q8-9-10 and Q12
MIXER 10 20
Q8-9-10 20 30
Q12 30 40
SYSVGAIN:2POR 10 40
END
FREQ
ESTP 20HZ 20KHZ 500
END

```

zero in on the radio's subsystems. To model how these blocks work together as one big system, we chain them end to end (using a new set of arbitrary node numbers that happen not to duplicate those of the constituent blocks) in a fourth netlist block:

```

BLK
MIXER 10 20
Q8-9-10 20 30
Q12 30 40
SYSVGAIN:2POR 10 40
END

```

ARRL Radio Designer will use this block to calculate the Ugly Weekender's overall audio voltage gain, so we'll name it SYSVGAIN.

Finally, we tell ARD to calculate these four blocks' performance at 500 exponentially stepped frequencies from 20 Hz to 20 kHz:

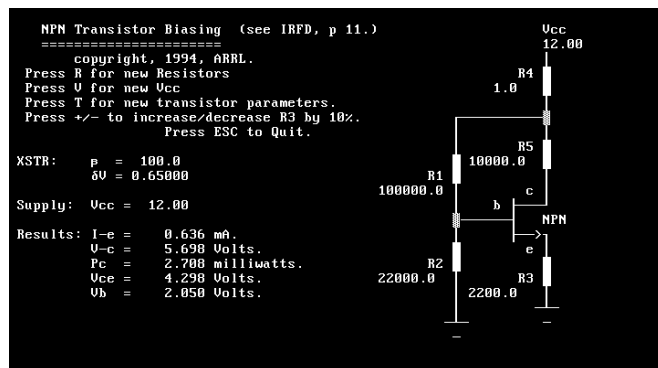


Figure 2—Wes Hayward's *NPNBIAS.EXE*, one of over 20 utilities on the software disk shipped with ARRL's reissue of his *Introduction to Radio Frequency Design* book, provided me with the collector currents coded into the BIP netlist lines in Table 1.

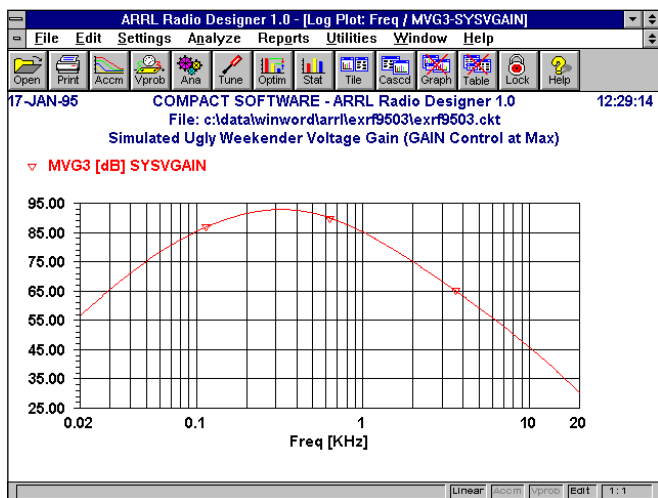


Figure 3—ARRL Radio Designer can evaluate voltage gain in several ways. This graph merely expresses in decibels the ratio of the UWR's output voltage (at the output terminals of the Q12 block, terminated in $2000 + j0 \Omega$, the circuit's anticipated headphone load) to input voltage (at the input terminals of the MIXER block, with MIXER's input terminated in $50 + j0 \Omega$)—just the sort of measurement we're implying when we say that a direct conversion receiver must have "80 to 100 dB" of audio gain. Our model predicts that the Ugly Weekender Receiver's maximum audio gain—without its optional LM386 audio power amp IC—is just under 94 dB at its passband peak.

```

FREQ
ESTP 20HZ 20KHZ 500
END

```

Transistors

The Table 1 netlist models the Ugly Weekender's four 2N3904 bipolar junction transistors (BJTs) with ARD's BIP element. Because our modeling goals are simple—we're pretty much after just gain and impedance, and at audio frequencies to boot—we can get away without specifying any more than the transistors' alphas and emitter resistances, which happen to be the only two BIP parameters ARD absolutely can't live without.

An alpha of 0.99—corresponding to a beta of 100—is a safe assumption for a garden-variety small-signal transistor like the 2N3904. For emitter resistance (RE), we insert a formula $(26 [^{\circ}\text{C}, \text{ that is—room temperature}] \div \text{collector current in milliamperes})$ that comes from the well-established transistor model developed by Ebers and Moll. Yanked out of Table 1 and grouped, the UWR's four 3904s look like this:

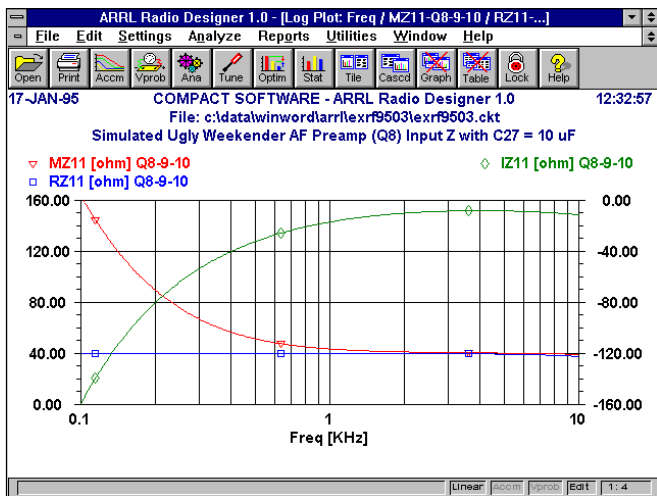


Figure 4—The Ugly Weekender's first audio preamp, Q8, is biased to terminate the radio's mixer with a resistive load reasonably close to 50 Ω across the span of common CW receiving pitches. *ARRL Radio Designer* can report complex impedances in magnitude and $R + jX$ (that is, real-imaginary) form, so this graph shows Q8's input impedance in terms of magnitude (MZ_{11} , triangular marker, 0 to 160- Ω scale), resistive component (RZ_{11} , square marker, 0 to 160- Ω scale) and imaginary (reactive) component (IZ_{11} , diamond marker, 0 to -160- Ω scale).

BIP 13 12 11 A=0.99 RE=(26/0.636); Q8
 BIP 12 14 15 A=0.99 RE=(26/0.5); Q9
 BIP 14 18 16 A=0.99 RE=(26/0.886); Q10
 BIP 42 43 0 A=0.99 RE=(26/1.919); Q12

We could hand-calculate the transistors' collector currents, of course—a drag—or snip our own Ugly Weekenders' 2N3904 collector leads and measure the currents (highly recommendable to keep your modeling on the rails, if you can take the time). Short on time, I instead used *NPNBIAS.EXE* (Figure 2), one of the 20+ utility programs included on the software disk shipped with ARRL's hot-off-the-press reissue of Wes Hayward's *Introduction to Radio Frequency Design* book, to give me ballpark collector-current numbers.

Modeling Results

Figures 3, 4, and 5 tell the rest of the story. Our Ugly Weekender Receiver model predicts realistic audio gain—just below 94 dB, max—and a useful degree of AF bandwidth limiting. And Q8, the UWR's post-mixer preamp, does indeed exhibit a reasonably resistive input impedance that's reasonably close to 50 Ω across the audio range of interest.

Now you'll have to excuse me—this modeling has got me so revved up about the Ugly Weekender that I think I'll go contact somebody with the real thing!

Free Electronic Goodies

You can get expanded versions of the Table 1 netlist in the file EXRF9503.TXT, available from the ARRL HQ BBS (203-666-0578) and Internet info server (info@arrl.org). As always, the latest *ARRL Radio Designer* news is also available from those sources in the file ARD.TXT.

Notes

¹David Newkirk, WJ1Z, "Introducing ARRL Radio Designer: New Software for RF Circuit Simulation and Analysis," *QST*, Oct 1994, pp 21-26.

²Seven circuits, but eight examples. *ARRL Radio Designer's* Example 2 is just Example 1—the 20-meter filter—with at least one error intentionally added to each of its netlist lines to give you experience in debugging a buggy ARD circuit file.

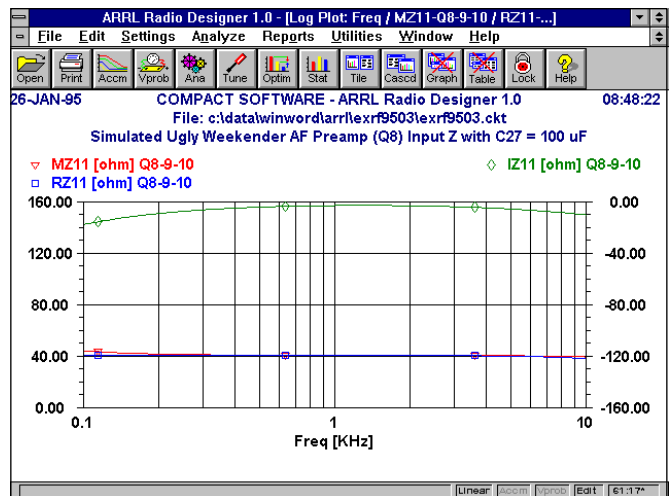


Figure 5—Finally, in five acts, a "what if?" detective vignette from the tattered casebook of Inspector ARD: (1) If Q8's input impedance is supposed to be flat and resistive across our audio range of interest, how come its magnitude (Figure 4's MZ_{11} curve) takes off so noticeably below 1 kHz? (2) Hey, the imaginary part of Q8's input impedance—Figure 4's IZ_{11} curve—pretty much tracks the reciprocal of that MZ_{11} rise over the same range! (3) Hmm, that IZ_{11} curve shows only negative values. By convention, we ascribe negative numbers to capacitive reactance, so the culprit must be a capacitor between the circuit's input terminal and Q8's emitter. (4) That would probably be C27, shown as 10 μ F in Figure 1. Come to think of it, 10 μ F does seem a bit small for a part that's supposed to act as a low-Z series element down to a few hundred hertz. So what would happen if we made C27 100- μ F instead? (5) Answer: a Q8 input impedance that's almost purely resistive from 100 Hz to 10 kHz. Case closed!

³I used the adjective *heterodyne* here to give us an imperfect start in differentiating among the various types of direct-conversion receivers, which have actually always included "crystal radios" and regenerative receivers (oscillating or not). Any receiver that converts RF directly to audio is a direct-conversion receiver, our tradition of using the term more narrowly notwithstanding.

⁴Roger Hayward, KA7EXM, "The 'Ugly Weekender' II: Adding a Junk-Box Receiver," *QST*, Jun 1992, pp 27-30.

New Products

PC PakRatt 2.0 AIMED AT WINDOWS POWER USERS

◇ *PC PakRatt for Windows 2.0* from Advanced Electronic Applications is a full-featured Windows-based control program that's aimed at power users of AEA's entire line of multimode data controllers (PK-900, DSP-1232, DSP-2232, PK-88 and the PK-232MBX). Version 2 also supports the PK-96 1200/9600-bit/s packet controller and the PK-12 1200-bit/s packet controller.

All *PakRatt* features are just a mouse-click away: Run two multimode controllers at the same time; try HF and VHF packet, AMTOR, Baudot RTTY, Morse code, ASCII, NAVTEX, PACTOR (with ANSI graphics); or let your multimode box figure out what that mysterious signal is (signal analysis mode).

Other features: simplified mail-drops; file transfers; split-screen operation; macros; built-in QSO logging; compatibility with AEA's *Log Windows 2.0* software; and more. Requirements: At least one of the above-mentioned AEA multimode data controllers and an IBM-compatible computer running Microsoft Windows.

Price: \$129; available from AEA or your local Amateur Radio dealer. Upgrades for users of version 1.0 are available from AEA. Contact Advanced Electronic Applications at PO Box C2160, Lynnwood, WA 98036; tel 206-774-5554, fax 206-775-2340.

Math in a Box, Transistor Modeling, and a New Meeting Place

Someone told me that each equation I included in the book would halve the sales. I therefore resolved not to have any equations at all. In the end, however, I *did* put in one famous equation, $E = mc^2$. I hope that this will not scare off half of my potential readers.

—Steven J. Hawking, in acknowledgments to his book *A Brief History of Time*

The TV you watch, or the radio transceiver which with you talk to the world, is a math machine—math put to work in the form of a kinetic sculpture built from copper, carbon, plastic, silicon and steel. You feed radio (or audio) and ac-line energy—both of which can be readily described numerically—in. The TV or radio performs mathematical operations on those inputs, and a desired, quantifiable result comes out: a sound, a moving picture, a declaration of rook to queen's bishop three from a chess opponent in Spain.

Because radio-electronic circuitry “just does math,” math can predict and analyze the action of radio-electronic circuitry. Until recently, enjoying the fruits of heavier-duty radio math pretty much led to two choices: become fluent in enough math to do your own designing, or be content with someone else's designs. The first choice leads smack into the field of radio engineering, which isn't for everyone. The second choice amounts to little more than just consuming radio products engineered by someone else—another entirely acceptable approach to a radio hobby. Crossing from designer to user is relatively easy—just relax and float downstream, preferably without tools—but moving from user toward designer generally involves a growing struggle with math.

Math is actually a *language*—invariably a *second* language—that's greatly more abstract than any mother tongue. The snag is that, by tradition, the math education we receive in our formative years rarely reflects this fact. Our introduction to numbers, counting, addition, subtraction, multiplication and division proceeds smoothly (one might even say naturally) enough. But for many of us—probably most, if Steven Hawking's aside about equations in *A Brief History of Time* is a trustworthy indicator—a point eventually comes beyond which we just don't “get” how some string of alien, inorganic gibberish (say, $y = mx + b$) can possibly mean what its English supporting text says it means (“the definition of a line”). Remedial instruction fails, we may get another dose of that slow-acting poison—“*anyone* can do this if they *really* try” (struggling Morse code learners, does this ring a bell?)—real life or an acceptable substitute beckons, and we bolt for the door.

However sordid our educational pasts, many of us survive Amateur Radio license exams, slog our way through to a license, and find this hobby a blast. Some of us—by no means all of us—discover in ourselves that fiddling with what's *inside* a radio box can be quite a kick. Just building radios from scratch or kits may satisfy this discovery. To others, a little reading here and a bit of tentative calculator poking there—successfully scaling a 223-MHz antenna to 440 MHz, say—may suggest that the time for Peace Talks with Math may have come.

Suddenly, math's prickly language seems to be talking about things that *count*: deepening a TVI filter's stopband loss until the neighbors resume smiling, or flattening out an amplifier's phase response so it doesn't cream 9600-baud data. Now math's alien tongue refers to ideas we can wrap our minds around—problems that get past our math antibodies because *they're not just made up* like math-textbook examples. It's still tough, because it's just not our thing and probably never will be. But now we're going to dig

in and get whatever value out of it we can.

This is a terrific time to make peace with math because this is the time of the personal computer. We use computers to do so many things that it's easy to forget that *all they ultimately do is math*. Whatever a computer does—e-mail, processing words, balancing a checkbook, stress-analyzing an aircraft wing, wafting you on a cybertrip down a virtual Amazon—it does by crunching numbers. If you can represent a thing or the action of a thing numerically—that is, if you can *model* it mathematically—you can simulate it and its action with a computer.

Since radio-electronic circuitry just does math, a computer can readily simulate, predict and analyze the action of radio-electronic circuitry.¹ What makes a computer a vastly more powerful tool for doing this than a slide rule or a simple calculator is that it can *contain* math in addition to just solving it. You can use a computer to determine, say, a coil's reactance (ac resistance) even if you know *absolutely nothing* about how to do it with math. All you do is run a program that contains the right equation(s) and lets you interact with them in a suitably friendly way. You tell the computer your coil's inductance and the frequency at which you need to know its reactance, and the computer tells you the answer.

This is why we're so excited about putting *ARRL Radio Designer*² into the hands of radio hobbyists. Quite a few hams would like to tinker with the nuts and bolts of radio-electronic circuitry in ways that their modest math fluency just doesn't allow. They know many of the issues, concepts and quantities involved in RF and electronic design, but aren't comfortable with manipulating them numerically. For them—as well as for math-fluent hams who want to spend less time doing busywork—*ARRL Radio Designer* is a natural.

I'm not saying that *ARRL Radio Designer* just hands over the keys to the radio-engineering castle for \$150 plus shipping. What the program *does* do is stuff a large array of muscular radio-electronics math—math you can use regardless of how well you can speak it—into an affordable package. The more you already happen to know about radio and electronics design, the faster and farther you'll be able to go with *ARD*. Backing up your *ARRL Radio Designer* explorations with reading in solid textbooks like *ARRL's* reissue of Hayward's *Introduction to Radio Frequency Design* can brighten your light even faster.

More on Transistor Modeling in *ARRL Radio Designer*, Part 1

If modeling a circuit required us to *exactly* simulate the performance of the real thing, circuit simulators wouldn't be the major industrial and educational tools they are. Simulating even something as simple as a one-transistor 2-meter preamp would be so complex and painful that it wouldn't be worth doing. Luckily, much as the NTSC TV system can simulate moving pictures by tossing 525-line stills at us at the rate of 30 per second, simulated circuit behavior can be considerably grainier than a circuit's real-life performance and still return news we can use.

For example, in simulating a direct-conversion receiver's audio channel in March 1995 *Exploring RF*, we used *ARRL Radio*

¹It gets better: Since radio-electronic circuitry just does math, and a computer just does math, a computer—properly configured and programmed, of course—can also do the *work* of radio-electronic circuitry. We call this *digital signal processing*, and it's an increasingly big deal.

²Available from ARRL Publications Sales for \$150 plus shipping. See the ad elsewhere in this issue.

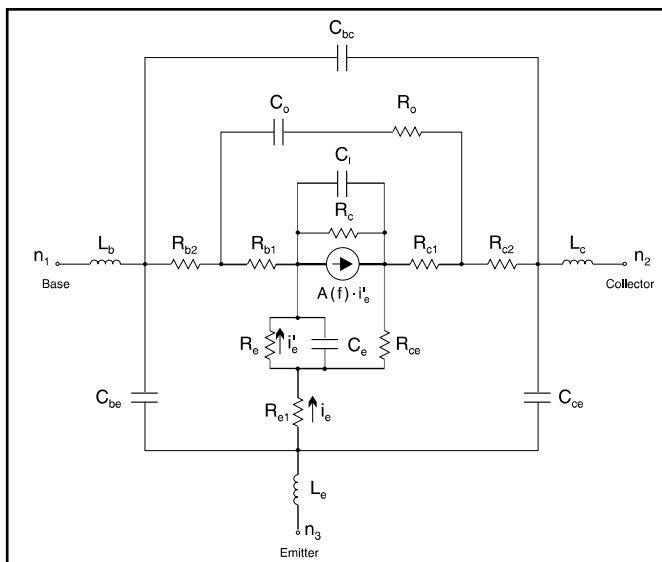


Figure 1—ARRL *Radio Designer's* bipolar junction transistor (BJT) element allows you specify a BJT's internal structure in great detail. You can safely accept *ARD's* defaults for many of these values when modeling at audio and low radio frequencies, but supplying true-to-life detail for accurate modeling at higher frequencies is a challenge in simulating with *ARD*. Most of the device characteristics listed in transistor databooks can't be directly plugged into this model!

Designer's BIP (bipolar junction transistor) element to supply only the roughest possible sketches of the circuit's four 2N3904 transistors.³ But if you're already been thumbing through *The ARRL Radio Designer Manual*, you know that the BIP element can handle much more detail. Take a deep breath and look at Figure 1.

Precise active-device modeling lies at the heart of any circuit simulator worth using, so a radio engineer might wade right into the complexity of *ARD's* BIP model with a bleat of pure joy. But many of us will wonder how we're ever going to model anything much more complex than a π attenuator if Figure 1 is any indication of what we'll need to succeed. A quick glance at the nearest transistor data manual seems to seal our doom: Of all the data listed for a particular type, almost none of it—if any—consists of numbers than we can plug directly into an *ARD* netlist!

Some circuit simulators—some of the *SPICE* variants come to mind—approach this problem by providing *device libraries* containing pre-coded device parameters. You select the part you

³We—uh, that is, I—also supplied a first-rate howler when I wrote in that column that the 26 in the emitter resistance formula ($R_E = 26/I_C$) conveys “°C, that is—room temperature.” Since that formula is just a variation on Ohm's Law, the 26 actually means 26 millivolts, the room-temperature value of V_T , which is the thermal equivalent of voltage in the transistor's semiconductor material.

Table 1

Two-Port S-Parameter Data Equivalent to an MRF586 Transistor (Operating at $V_{CE}=10$ and $I_C=90$ mA) from the ARRL *Radio Designer* Databank File MOTOROLA.FLP

100MHZ	.36	-123	15.68	107	.05	57	.44	-77
300MHZ	.33	180	5.78	83	.1	61	.32	-117
500MHZ	.34	154	3.44	70	.15	59	.39	-122
1000MHZ	.31	118	1.84	43	.25	51	.49	-133

Each dataset line conveys frequency and the S parameters MS_{11} (magnitude of S_{11}), PS_{11} (phase of S_{11}), MS_{21} (magnitude of S_{21}), PS_{21} (phase of S_{21}), MS_{12} (magnitude of S_{12}), PS_{12} (phase of S_{12}), MS_{22} (magnitude of S_{22}) and PS_{22} (phase of S_{22}).

want—by its actual part number, such as 2N5179, 2N3819 or 1N914—from a menu, and the program duly adds the corresponding code and device parameters to your netlist.

ARRL Radio Designer lets you do something a little different. Instead of device libraries, it comes with *device databanks*—manufacturer-supplied ASCII files that convey, in *S-parameter* (scattering parameter) form, the *actual measured performance* of RF devices operating under specific conditions. You can use this data directly or indirectly. The direct method involves plugging the databank data directly into *ARD's* two-port [TWO] black-box element and using that TWO instead of a BIP or FET. The indirect method makes use of databank data and *ARD's* optimization engine to adjust the values of a BIP or FET's many parameters until its performance matches the measured performance of an actual device as reflected in databank data.⁴

S-Parameter Basics

S parameters reflect a standardized way of characterizing how a device behaves in response to signal energy applied to its *ports*—its signal inputs and outputs—usually with all of its ports terminated in identical, standard impedances (commonly, 50 Ω resistive). A transistor, for instance, is a two-port device. By convention, the ports are labeled with numbers, Port 1 being the input and Port 2 being the output.

Signal energy applied to one port of a two-port device comes out two places: at the same port (the device reflects some of the energy back to the generator) and at the other port. How much signal comes out relative to the applied signal tells us the device's gain (which can be negative; that is, a loss); how much signal reflects back out tells us something about the impedance match between that port's impedance and our signal generator. Determining the phase of the output or reflection signals relative to the phase of the applied signals tells us even more about the device under test.

Figure 2 shows this idea graphically. An S parameter is a voltage ratio (commonly, but not always, expressed in decibels) annotated with two subscript numbers that indicate the ports involved. For instance, a device's forward transmission gain, S_{21}

⁴Although I've been mentioning only S parameters, *ARD* can also handle TWO data in Y or Z-parameter form.

Table 2

Two-Port S-Parameter Data Equivalent to a 2N3553 Transistor (Operating at $V_{CE}=28$ and $I_C=30$ mA) from the Motorola File 2N3553A.S2P

0.045	0.76	-165	10.39	96	0.03	38	0.23	-87
0.046	0.75	-166	10.09	95	0.03	46	0.23	-87
0.047	0.77	-166	9.84	94	0.03	48	0.22	-88
0.048	0.75	-166	9.78	94	0.02	44	0.22	-87
0.049	0.77	-166	9.67	94	0.03	45	0.22	-89
0.05	0.76	-167	9.39	94	0.03	51	0.21	-88
0.051	0.78	-168	9.23	93	0.03	51	0.21	-89
0.052	0.76	-168	9.04	93	0.03	49	0.21	-88
0.053	0.77	-167	8.92	92	0.03	52	0.2	-87
0.054	0.77	-168	8.7	92	0.03	44	0.21	-87
0.055	0.76	-168	8.66	92	0.03	49	0.21	-85

Like the .FLP-format data shown in Table 1, each of these lines conveys frequency and the S parameters MS_{11} , PS_{11} , MS_{21} , PS_{21} , MS_{12} , PS_{12} , MS_{22} and PS_{22} . The difference between this data format (that used by EESOF's *Touchstone* circuit simulator, and one of several S-parameter data formats now standard in the RF CAD community) and that of Table 1 is that an .S2P file conveys frequency in gigahertz but does not include frequency units. Because *ARD* assumes that you're talking hertz if you specify frequencies without units, you must add the suffix GHZ, with no leading space—0.045GHZ instead of 0.045 GHZ—to each frequency spec in any .S2P data you use in an *ARD* netlist.

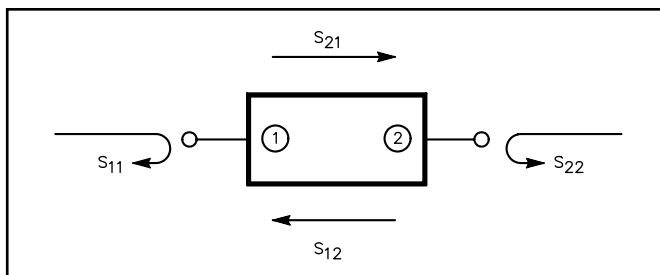


Figure 2—S parameters corresponding to input reflection, forward transmission gain, reverse transmission gain and output reflection can quite closely characterize a small-signal linear device—in this case, a *two-port* device. The text describes one way you can use *ARD*'s TWO element to insert manufacturer-supplied S-parameter data directly into your netlists.

("S sub two one"), is the ratio of the voltage at Port 2 to the voltage applied to Port 1—a value that must be expressed as a vector to convey the two signals' relative phase. To discuss S parameters more readily and to communicate them in tabular form, we split each of the four basic ones into separate magnitude and phase components: MS_{11} (magnitude of input reflection) and PS_{11} (phase of input reflection); MS_{21} (magnitude of forward transmission gain) and PS_{21} (phase of forward transmission gain); MS_{12} (magnitude of reverse transmission gain) and PS_{12} (phase of reverse transmission gain); and MS_{22} (magnitude of output reflection) and PS_{22} (phase of output reflection).

One big deal about S parameters is that they can cram a lot of very realistic information about a device's behavior into just a few standardized numerical values. If we know the S parameters of a given transistor operating under known conditions, we have a very useful picture of how it looks to the outside world—a picture we can paste directly into *ARRL Radio Designer*. This leads to another big deal about S parameters: RF-device manufacturers commonly make their products' S parameters freely available in forms that *ARD* can easily digest! Tables 1 and 2 show two very similar S-parameter formats that *ARD* can handle.

Plugging S-Parameters into ARRL Radio Designer

Using Table 1's data to synthesize a transistor in an *ARD* netlist is so straightforward that I won't even show you a complete netlist. Instead of connecting a BIP to nodes 3, 4 and 5, connect a TWO to those nodes in the same sense and give it a distinctive label:

```
TWO 3 4 5 MRF586
```

At the end of your file—after your *FREQ* block's *END* statement—add the TWO's S-parameter data in a *DATA* block:

```
DATA
MRF586: S
100MHZ .36 -123 15.68 107 .05 57 .44 -77
300MHZ .33 180 5.78 83 .1 61 .32 -117
500MHZ .34 154 3.44 70 .15 59 .39 -122
1000MHZ .31 118 1.84 43 .25 51 .49 -133
END
```

The data's header tags it as S-parameter information intended for use with an element labeled MRF586. Additional header information is necessary for Y or Z-parameter data, and for S-parameter data derived at impedances other than 50 Ω . (See pages 15-50 and 15-51 of *The ARRL Radio Designer Manual* for detail on the TWO element, and pages 17-7 and 17.8 of the *Manual* for how to specify data for TWO.)

This example is simplistic in that its *DATA* block doesn't include any numbers *ARD* can use to predict how a transistor-flavored TWO element may generate noise. We'll overcome this limitation in the next edition of *Exploring RF*.

ARRLCAD: A Meeting Place in Cyberspace

If you can send and receive Internet e-mail, you can now share your questions, answers, ideas and views about *ARRL Radio Designer* and other Amateur-Radio-related circuit and antenna design and simulation tools with other users. Thanks to the generous provision to ARRL of facilities and volunteer support by TAPR, Tucson Amateur Packet Radio Corp, I'm pleased to announce the startup of ARRL's computer-aided-design e-mail reflector, ARRLCAD.

You may already know of e-mail reflectors as *mailing lists*. A mailing list automatically sends copies of any messages sent in by its subscribers back out to all of its subscribers. It's somewhat like a UseNet newsgroup, the difference being that you don't need access to UseNet newsgroups to participate. All you need is the ability to send and receive Internet e-mail.

To subscribe to ARRLCAD, send an e-mail message to

```
listproc@tapr.org
```

with text that reads

```
subscribe arrlcad FirstName LastName
```

in which *FirstName* and *LastName* mean exactly that. (My subscription message read `subscribe arrlcad david newkirk`, for example.) The reflector software will confirm your subscription with a welcoming information message. Subscribing to ARRLCAD costs *nothing at all*.

ARRL Radio Designer Front-Line Support

ARRL HQ's Technical Information Service provides front-line support for *ARRL Radio Designer*. Just ask for the Technical Information Service at 203-666-1541 (fax 203-665-7531), or send email to tis@arrl.org.

Free Electronic Goodies

This month's circuit example is so straightforward that it doesn't warrant the creation of an EXRF9505.TXT file, but as a consolation prize I've added to the heap the file ARD9410.TXT, an ASCII-text version of the October 1994 *QST* article that introduced *ARRL Radio Designer*. To find out how to get it and the files ARD.TXT, EXRF9501.TXT and EXRF9503.TXT from the ARRL Technical Information Service's automated info server (info@arrl.org), send the server an e-mail message that reads just

```
INDEX
```

```
HELP
```

and you'll receive instructions by return e-mail. ARD.TXT, ARD9410.TXT, EXRF9501.TXT and EXRF9503.TXT are also available from the ARRL BBS (203-666-0578) and via FTP from the ARRL subdirectory at oak.oakland.edu. **QST**

New Products

HAND-MADE KEYS/PADDLES

◊ If a hand-made key that won't slide around and won't be damaged by a fall to the floor is in your future, consider the line of keys and paddles produced by Karl Reiber, N6NXM, of Bakersfield, California.

Karl's keys, made one at a time by special order, come in various sizes and configurations and can be made from steel, brass and aluminum. They feature highly polished and tested needle bearings for smooth operation. Each piece is guaranteed for three years. If he can't repair your key, N6NXM will build you a new one.

Contact Titan Communications (ask for Jerry, KD6EAG) at 805-399-1321, or the designer, Karl Reiber, N6NXM, 446 Sycamore Dr, Bakersfield, CA 93308; 805-393-5855. **QST**

Transistor Modeling with ARRL Radio Designer, Part 2: Optimization Produces Realistic Transistor Simulations

As we saw in May 1995 *QST*'s Exploring RF, modeling a transistor's behavior merely by shoving its measured S, Y or Z parameters into *ARRL Radio Designer*'s two-port black-box element (TWO) is a neat thing to be able to do because it's so simple. All you do is link valid, transistor-flavored two-port data—a considerable body of which ships with each copy of *ARRL Radio Designer* in the form of manufacturer-supplied databank files, and more of which are ongoingly available from transistor manufacturers—into a TWO, and the black box acts like the transistor represented by the data.

Well, almost. Modeling transistors with the black-box-and-data approach involves a few limitations. For one thing, if you don't have S, Y or Z-parameter data for the transistor you want to model, you can't model it with a black-box element. Because manufacturers tend make data available only for relatively recent transistors intended for RF use, many garden-variety transistors don't show up in databank form. In other words, don't get your hopes up about finding manufacturer-supplied S-parameter data for, say, a 2N3904.

Even if you can find data for your device of interest, it may not be usable at your frequency of interest. For example, the *ARD* databank file MOTOROLA.FLP includes S-parameter data for an MRF581 operating at a collector current of 100 mA and with 10 V between its collector and emitter—

```
300MHZ .66 -172 8.46 93 .05 49 .3 -134
500MHZ .68 174 5.06 82 .07 56 .25 -147
1000MHZ .68 157 2.64 65 .12 64 .23 -172
1500MHZ .72 139 1.86 52 .17 63 .27 -177
```

—but only for 300, 500, 1000 and 1500 MHz. That none of these frequencies corresponds to a ham band is no problem: If a data set includes parameters for at least three frequencies, *ARRL Radio Designer* can interpolate parameter values for frequencies between them. All you need to do is specify the additional frequencies in your **FREQ** block.

A major limitation of databank data is that it cannot be reliably extrapolated—that is, used as a basis for educated guesses about device performance *outside* a data set's frequency span. It so happens, for instance, that many bipolar junction transistors intended for CATV service are just what the doctor ordered for strong, low-noise, linear amplifiers at HF. A broadband HF amplifier using a push-pull pair of MRF581s standing at 100 mA each would be a fine thing to model with *ARD*—but the lowest frequency for which Motorola supplies MRF581 S parameters is 300 MHz! What's a modeler to do?

Use *ARRL Radio Designer*'s BIP—bipolar junction transistor—model, that's what! As I wrote in the previous (May 1995) Exploring RF, however, a person just can't be expected to flesh out an *ARRL Radio Designer* BIP element—or an *ARD* FET for that matter, because it's just about as complex—by poring through transistor databooks and making educated guesses. The numbers we need to model transistors accurately just plain don't live in those books. Unless we're intimately involved with engineering and manufacturing transistors, we probably can't even begin to guesstimate appropriate starting values for a real transistor's tiny internal Rs, Cs and Ls—factors that are nonetheless crucial in modeling transistor behavior from the shortwaves on up.

The great news is that we don't have to guess at these parameter values. If we have valid S, Y or Z-parameter data for a given transistor, we can use *ARRL Radio Designer*'s optimizer to adjust a BIP's (or FET's) parameters so its behavior matches the data. The result is a realistic, databank-data-derived transistor model that can be used outside the frequency limits of the data used to generate it!

From July 1995 *QST* © 1995 ARRL

Table 1

ARRL Radio Designer Netlist for an Optimized BIP

```
BLK
  BIP 1 2 0 A=0.98 RE=?1? F=?1GHZ? T=?0.01NS?
  + CE=?0.1PF? CI=?1PF? RC=?1000? R0=?0.1? C0=?1PF?
  + RB1=?1? RC1=?1E-6? RB2=?0.1? CBE=?10PF?
  + CBC=?0.1PF?
  + CCE=?100E-6PF?
  + LB=?1E-6NH? LC=?100E-6NH? LE=?0.1NH?
  OPTBIP:2POR 1 2 0
END
BLK
  TWO 1 2 0 MRF581
  REALBIP:2POR 1 2 0
END
FREQ
  300MHZ 500MHZ 1000MHZ 1500MHZ
END
OPT
  OPTBIP S=REALBIP
  TERM=0.01
END
DATA
  MRF581:S
  * MOTCL MOTOROLA MRF581 Vce=10V Ic=100mA
  300MHZ .66 -172 8.46 93 .05 49 .3 -134
  500MHZ .68 174 5.06 82 .07 56 .25 -147
  1000MHZ .68 157 2.64 65 .12 64 .23 -172
  1500MHZ .72 139 1.86 52 .17 63 .27 -177
END
```

Data + Optimization = Realistic BIP

Doing this with *ARRL Radio Designer* is easier done than said. All we need is a netlist that includes:

- a **BLK** block containing the BIP element we want the optimizer to adjust
- a **BLK** block containing a **TWO** element that *ARD*'s optimizer can use as a standard of comparison in adjusting the BIP
- a **FREQ** block that specifies only the frequencies represented in our **TWO**'s S (or Y or Z) parameter data
- an **OPT** block that tells *ARD*'s optimizer to compare the performance of the BIP's **BLK** to the performance of the **TWO**'s **BLK** and tweak the BIP's parameters so their performances closely match; and
- a **DATA** block containing the S (or Y or Z) parameter data for the actual transistor we want our BIP to simulate.

Table 1 shows a ready-to-optimize netlist based on the MRF581 data presented a few paragraphs ago. Let's examine some of its finer points block by block.

The first block—

```
BLK
  BIP 1 2 0 A=0.98 RE=?1? F=?1GHZ? T=?0.01NS?
  + CE=?0.1PF? CI=?1PF? RC=?1000? R0=?0.1? C0=?1PF?
  + RB1=?1? RC1=?1E-6? RB2=?0.1? CBE=?10PF?
  + CBC=?0.1PF?
  + CCE=?100E-6PF?
  + LB=?1E-6NH? LC=?100E-6NH? LE=?0.1NH?
  OPTBIP:2POR 1 2 0
END
```

—contains the BIP we'll adjust for equivalence with databank data. All of its parameters are optimizable—question marks delimit optimizable values—except A (the BIP's alpha). (We must keep the optimizer's mitts off of alpha because it immediately attempts to raise alpha to a value greater than 1—impossible for a real transistor—and halts itself with a suitably pithy error message. [Try putting question marks around the A entry's 0.98 value when you optimize this netlist, and you'll see what I mean.]

Using the example of page 15-10 of *The ARRL Radio Designer Manual* as a guide, I've preloaded all of the BIP's optimizable parameters with nonzero values (zero-value parameters instantly choke the optimizer by violating the rules of floating-point math) at realistic orders of magnitude. Preloading at realistic orders of magnitude speeds optimization. Preloading values at wildly inappropriate orders of magnitude (10 farads for CE and 1 megohm for RB1, for instance) may simulate a transistor that's essentially broken from the gitgo, forcing the optimizer to dig itself out of such a deep hole that success may be impossible.

The + signs tell *ARD* to ignore hard carriage returns within the BIP specification. This lets us specify a BIP's many parameters without extending one long netlist line far off the edge of our screen.

We code our optimizable BIP into a circuit all by itself. For convenience, I've given this circuit the name OPTBIP because it contains the BIP we're going to optimize. (Any syntax-valid name is fine as long as we use the same one here and in the "circuit to optimize" part of our OPT block's first command line.)

The second block—

```
BLK
  TWO 1 2 0 MRF581
  REALBIP:2POR 1 2 0
END
```

—contains the TWO element to which *ARD*'s optimizer will compare the performance of the optimizable BIP in our OPTBIP circuit.

We code our referent TWO into a circuit all by itself. For convenience, I've given this circuit the name REALBIP because it's our means of injecting real bipolar-transistor performance, in the form of databank data, into this optimization. (There's nothing sacred about the name OPTBIP; any syntax-valid string is fine as long as we use the same name in the "goal" part of our OPT block's first command line.)

The MRF581 at the end of our TWO's netlist line is a *label* that lets us link databank data (contained in the DATA block and introduced by the same label) with this TWO. (The label could be any syntax-valid string; MRF581 is appropriate because we happen to be modeling one this time around.)

The third block—

```
FREQ
  300MHZ 500MHZ 1000MHZ 1500MHZ
END
```

—tells *ARRL Radio Designer* the frequencies at which we want to model (and optimize) the behavior of OPTBIP and REALBIP. This block's four frequencies mirror those of the MRF581 databank data.

The fourth block, the optimization block—

```
OPT
  OPTBIP S=REALBIP
  TERM=0.01
END
```

—tells *ARD*'s optimizer what to do. The first command line—

```
OPTBIP S=REALBIP
```

—specifies our optimization goal, telling *ARD* to make the S parameters of the circuit called OPTBIP equivalent to those of the circuit called REALBIP. (I've kept this goal specification simple by not specifying *weighting*, through which we can tell *ARD* how much each of the optimizable circuit's S parameters [for a two-port network, S_{11} , S_{21} , S_{12} and S_{22}] contributes to the optimizer's

error function. All four weights therefore default to 1.)

The next line—

```
TERM=0.01
```

—tells *ARD* to terminate optimization when the optimizer's error function falls to 0.01. (If we see that the process has reached the point of diminishing returns before TERM has been reached, we can always abort the optimization and accept whatever results we have.)

This OPT block doesn't specify optimization frequencies, so the optimizer uses the frequencies specified in the FREQ block.

Finally, the DATA block—

```
DATA
  MRF581:S
  * MOTCL MOTOROLA MRF581 Vce=10V Ic =100mA
    300MHZ .66 -172 8.46 93 .05 49 .3 -134
    500MHZ .68 174 5.06 82 .07 56 .25 -147
    1000MHZ .68 157 2.64 65 .12 64 .23 -172
    1500MHZ .72 139 1.86 52 .17 63 .27 -177
END
```

—contains S-parameter data for a TWO element labeled MRF581—the TWO in the REALBIP block. This is the data to which the optimizer ultimately compares the optimizable BIP's performance during optimization.

Running It

Now comes the fun part. With the Table 1 netlist loaded into *ARRL Radio Designer*, click *ARD*'s **Optim** button or press **Shift+F10**. *ARD* analyzes the netlist and then pops its Optimization and Optimization Data dialog boxes. In the Optimization dialog, enter 100 as the number of iterations, change the optimization type to Gradient, clear the Display check box (a good habit to get into because turning off Display greatly speeds computations if you have any graphs onscreen), click Optimize (or press **Alt+O**), and you're off!

In just a few iterations, the optimizer pushes the error function below 0.2 and reports that the gradient has become too small for the process to continue. True, 0.2 is considerably higher than the TERM value we specified (0.01), but perhaps OPTBIP's performance already resembles that of REALBIP closely enough for whatever modeling we want to do. Take a look at Figures 1 and 2 and judge for yourself. It looks to me like OPTBIP's performance already falls comfortably within the sample-to-sample variation we'd expect in a handful of off-the-shelf MRF581s.

Getting Closer with Single-Frequency Modeling

We've just done a pretty good job of making an *ARD* BIP exhibit the gain and I/O-reflection behavior of a real transistor—over the frequency range (300 to 1500 MHz) covered by that transistor's databank data. Because the optimized BIP owes its performance to (among other things) the realistic internal resistance, capacitance and inductance values written into it by the optimizer, it's frequency-transportable in a way the REALBIP TWO's databank data isn't. We can now quite confidently use the optimized BIP at, say, 30 MHz—far below the frequency represented in the data set we used to optimize it!

If we want to use our optimized BIP *only* at frequencies below the data set's lowest frequency, we can improve our results by making the optimizer concentrate all of its number-crunching power at the lowest frequency in the data set. To do this, we just comment out the FREQ block's 500MHZ, 1000MHZ and 1500MHZ entries by inserting a semicolon between 300MHZ and 500MHZ, and add a leading asterisk to the corresponding lines in the DATA block's MRF581 data set.

Table 2 shows the results of doing this with our MRF581 data. The performance of OPTBIP and REALBIP now match much more closely at 300 MHz (error function ≈ 0.03) than we were able to achieve with the optimizer working to equalize their behaviors over the 300 to 1500-MHz range. Now we can get to work modeling that push-pull MRF581 amplifier at 14 MHz!

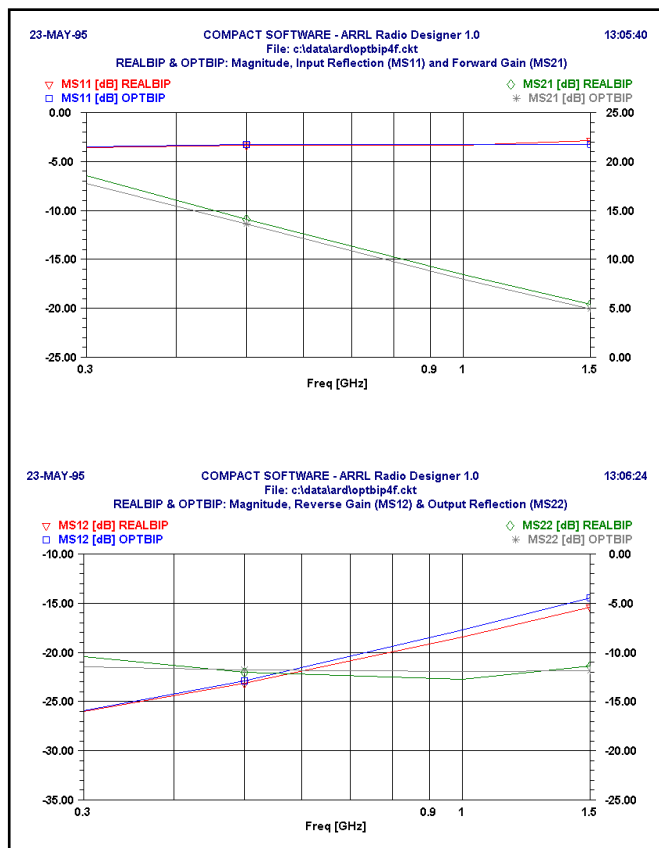


Figure 1—Optimized for a performance match with the measured S parameters of a real MRF581 transistor (REALBIP traces), the performance of an *ARRL Radio Designer* bipolar-junction transistor model (OPTBIP traces) mirrors the real thing to within a decibel. These graphs compare the magnitude components of the real and simulated transistors' S parameters; Figure 2 compares the phase components of both transistors' S parameters. (What's an S parameter, anyway? See Exploring RF for May 1995.)

What About Optimizing an ARD FET?

Now that I've shown you how to optimize a BIP on the basis of S parameters, you're ready to try a FET. *ARRL Radio Designer's* Example 4, a simple JFET preamp, contains a FET element that's already optimized on the basis of the measured Y parameters of a 2N4416. Your job? Copy the Example 4 netlist into a new file and optimize an unoptimized FET element using the Y parameters in EXAMPLE4.CKT's DATA block.

Transistor Modeling with ARD, Part 3

Optimization is one of the two biggest smile-producing features *ARRL Radio Designer* brings to affordable RF CAD, and accurate noise modeling is the other. This month, I've shown how you can crank a transistor's S, Y or Z parameters through ARD's optimizer to make an ARD BIP or FET exhibit the same parameters, but space hasn't allowed me to touch on the complex issue of modeling noise with TWOs, BIPs and FETs. That important transistor-modeling concern deserves special treatment, and it's coming right up—in September *QST's* Exploring RF.

Free Electronic Goodies

An expanded version of the Table 1 netlist is available (as the file EXRF9507.CKT) from ARRL HQ's Internet info server (send e-mail to info@arrl.org with a message text consisting of just the word *HELP*). A ZIP file containing EXRF9507.CKT and its accompanying *ARRL Radio Designer* report file (EXRF9507.RP2) is available from the HQ BBS (203-594-0306) as EXRF9507.ZIP, and via FTP (as [exrf9507.zip](ftp://exrf9507.zip)) from the pub/hamradio/arrl directory

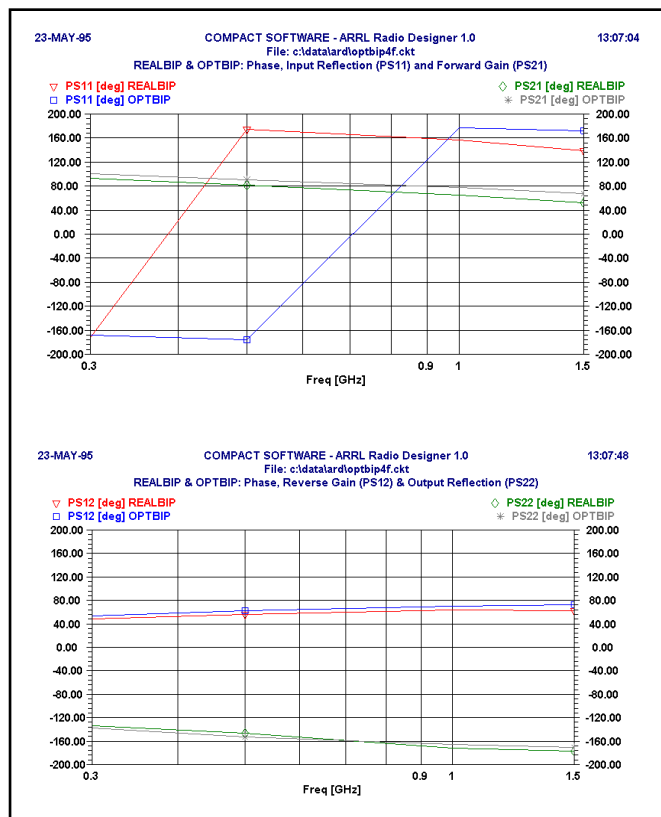


Figure 2—The phase performance of a real MRF581 transistor (REALBIP traces) and an *ARD* transistor model optimized to act like it (OPTBIP traces) coincides closely. The steep transitions in the PS₁₁ and PS₂₁ traces—from -180° to +180°—are artifacts of graphing these quantities rectangulary (-180° and +180° are actually the same point) with only four data points, so the PS₁₁ and PS₂₁ performance of REALBIP and OPTBIP track more closely than the wide horizontal gulf between the transition lines indicates to the eye.

Table 2

300-MHz S Parameters of OPTBIP and REALBIP (OPTBIP Optimized at 300 MHz)

	MS ₁₁ (dB)	PS ₁₁ (°)	MS ₂₁ (dB)	PS ₂₁ (°)	MS ₁₂ (dB)	PS ₁₂ (°)	MS ₂₂ (dB)	PS ₂₂ (°)
REALBIP	-3.61	-172.0	18.55	93.0	-26.02	49.0	-10.46	-134.0
OPTBIP	-3.23	-167.9	17.98	98.2	-26.31	53.6	-10.23	-132.9

at oak.oakland.edu.

ARRLCAD, an e-mail reflector (mailing list) through which you can now share your questions, answers, ideas and views about *ARRL Radio Designer* and other Amateur-Radio-related circuit and antenna design and simulation tools with other users, is up, running and growing. To subscribe to ARRLCAD, send an e-mail message to

listproc@tapr.org

with text that reads

subscribe arrlcad FirstName LastName

—in which *FirstName* and *LastName* mean exactly that. (I subscribed with the message subscribe arrlcad david newkirk, for example.) The reflector software will confirm your subscription with a informational welcome message. Subscribing to and participating in ARRLCAD costs *nothing at all*. 

Transistor Modeling, Part 3: Constraints in Optimization, Two-Port Data with Noise Parameters, and Introducing ARRL Radio Designer 1.5!

In July 1995 Exploring RF, we did a pretty good job of optimizing an ARRL Radio Designer (ARD) bipolar junction transistor (BJT) element to make it act like an ARD two-port element (TWO)—a TWO characterized by manufacturer-supplied *databank* data for an actual transistor. This gave us a transistor model usable outside the frequency limits of the databank data used to generate it—a neat thing to be able to do, especially considering that manufacturers usually don't characterize today's small-signal RF transistors at frequencies below a few hundred megahertz.

Notice, though, that I said *pretty good*. I said that because I neglected to do one very important thing in running that BJT optimization: Believing I had determined that the optimizer could not be kept from immediately driving the BJT's alpha (A) to a value equal to or greater than 1 (illegal for a real transistor) if A were made optimizable ($A = ? .99?$), I hard-coded A to a realistic, yet unoptimizable, value (0.98) and went on to achieve the results shown in last column's Figures 1 and 2.

But it turns out that an A of 0.98 was my lucky guess. A transistor's gain is one of its most important characteristics, so what I *should* have done with A was make it optimizable and *constrain*—set hard limits for—the range of A values the optimizer could try in finding its solution. Doing so would let the optimizer adjust A for best results while keeping its value realistic.

We can constrain optimizable ARD values in several ways. An *unconstrained* value takes the form ?NominalValue?—one value bracketed by question marks—as in $R = ?50K\Omega?$, or $A = ? .99?$. During gradient optimization, ARD can replace NominalValue with any value from 0.0 to infinity; during random optimization, ARD can replace NominalValue with any value from 0.0 to no more than twice NominalValue—a range that's considerably smaller than that available during gradient optimization. Either way, we now can see why specifying a BJT's optimizable alpha as $A = ? .99?$ could result in unrealistic values of A: During gradient optimization, A's upper limit would be infinity; during random optimization, A's upper limit would be 1.98. Since A must be less

than 1 in any real transistor, both values are illegal. Luckily, ARD's BJT modeling is too smart to let us get away with either!

A *constrained* value takes the form ?Value NominalValue Value?, as in $C = ?10PF 100PF 500PF?$. ARRL Radio Designer handles this spec the same way in random and gradient optimization: Any value from 10 to 500 pF, inclusive, is fair game, and the game starts with the nominal (100 pF). As a variation on this, *relatively constrained* values take the form ?Value% NominalValue Value%?, as in $R = ?50\% 50 300\%?$, which specifies a resistance from 25 to 150 Ω , inclusive, starting with 50 Ω . ARD handles relatively constrained values the same way in random and gradient optimization.

Unconstrained optimizable values may seem superficially simpler to use than constrained values, but they are actually trickier to use if we want to be able to switch smoothly back and forth between random and gradient optimization in solving a problem. (We may well want to do just this if gradient optimization bogs down in a local minimum, or if random optimization seems to be taking several lifetimes to zero in on its target.)

Just what do I mean by "trickier"? Random optimization with unconstrained values can fail to find a solution *if the solution happens to require that any of the optimizable values be increased to more than twice their nominal values*. If, for instance, we specify even one capacitor in an optimizable circuit as $C = ?100PF?$ and that capacitor's value for a successful solution must ultimately be 432 pF, we'll never get there with random optimization because ARD can't increase that unconstrained value to more than 200 pF. (We'd probably suspect something was up after noticing that ARD had stalled with this component's value right at 200 pF.) Switching to gradient operation could dig us out of the hole—if we didn't bog down in a local minimum, that is—because the gradient optimizer could increase the capacitor's value to infinity, as opposed to the 200 pF available to us in random optimization.

All things considered, if you're new to optimization with ARRL Radio Designer, it's probably best to use constrained rather

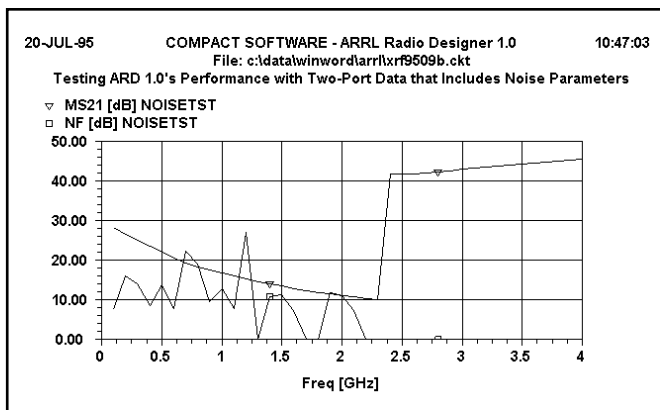


Figure 1—Graphing the performance of a two-port element (TWO) characterized by two-port data that includes noise parameters suggests that all is not well with active-TWO noise modeling in ARRL Radio Designer 1.0. No real, useful device would exhibit such extremes and fluctuations in forward transmission gain (MS_{21}) and noise figure (NF). Figure 2 shows how these traces *should* look.

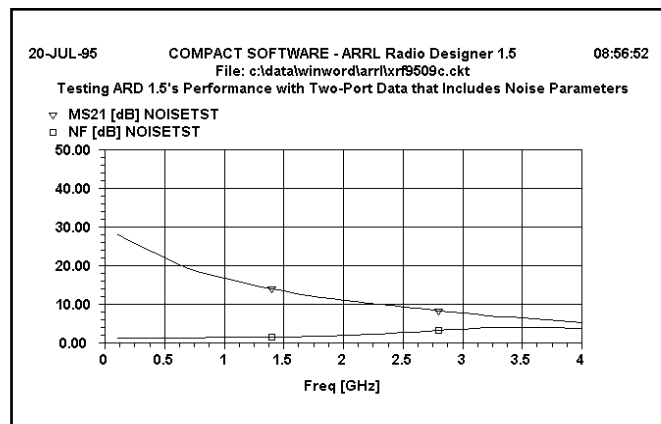


Figure 2—Now, *this* is more like it! Figure 1 shows unbelievable MS_{21} and NF values because of a bug in ARRL Radio Designer 1.0's routine for using noise-parameter data to model active-TWO noise. ARRL Radio Designer 1.5, now shipping to new users and available as a maintenance release for those already using ARD 1.0, solves the problem.

Table 1

Constrained Values for a More Optimizable BIP

```

BLK
BIP 1 2 0 A=? .85 .9 .999? RE=?0.5 1 15?
+ F=?0.1GHZ 1GHZ 10GHZ? T=?0.005NS 0.01NS 0.1NS?
+ CE=?0.01PF 0.1PF 5PF? CI=?0.005PF 1PF 10PF?
+ RC=?100 1000 10000? RO=?0.001 0.1 5?
+ CO=?0.001PF 1PF 10PF? RB1=?0.005 1 10?
+ RC1=?1E-7 1E-6 1E-5? RB2=?0.001 0.1 5?
+ CBE=?0.005PF 5PF 10PF? CBC=?0.001PF 0.1PF 5PF?
+ CCE=?1E-5PF 1E-4PF 1E-3PF?
+ LB=?1E-7NH 1E-6NH 1E-5NH?
+ LC=?1E-5NH 1E-4NH 1E-3NH?
+ LE=?0.001NH 0.1NH 1NH?
OPTBIP:2POR 1 2 0
END

```

than unconstrained optimizable values—constrained values specified to cover ranges that far exceed your reasonable expectations of what’s necessary for success. Then, you’ll be freer to switch between random and gradient optimization at will, and updating your circuit with the results of a run of either type won’t hobble subsequent runs of the other. (Using large *unconstrained* values doesn’t guarantee the same freedom, by the way. If, at some point during a series of optimizations, you tell the optimizer to update your circuit file with the latest values found by the optimizer, *ARD* may write at least one value that’s less than half of the value necessary for success. If this occurs, any subsequent random optimizations will stall short of solution until you modify the culprit value[s] by hand or switch to gradient optimization.)

Table 1 shows this constrained-value doctrine put to work. It’s just the OPTBIP circuit from July 1995 Exploring RF’s Table 1 modified to use constrained values.

It turns out that optimizing this constrained-values BIP with the same MRF581 data we used last time doesn’t give results that significantly differ from those achieved with $A = .98$ —that’s how good my guess for α was! In general, though, you’ll get better results in optimizing *ARD* transistor elements to behave like databank-data devices if you turn the optimizer loose on *every* parameter you use in your BIPs and FETs, suitably constraining them to keep your optimization on the rails.

Active-TWO Noise Modeling with ARRL Radio Designer

In the past two Exploring RFs, we’ve highlighted *ARRL Radio Designer*’s ability to use two-port S-parameter data in simulating transistor behavior. In the May column, we explored just plugging some basic MRF586 data into a netlist and calling the job done. In the July column, we learned how to optimize a BIP (or FET) on the basis of databank data—a principal value in this being that we gain a transistor model that’s frequency-transportable.

Using an optimized BIP or FET instead of the raw two-port data we’ve used so far does something else for us, though: It lets us use these transistor models’ ability to simulate noise.¹ Accurately simulating noise is important in radio design because successful radiocommunication requires that we keep winning the ongoing battle between signal and noise.

ARD can’t use the basic parameters-versus-frequency databank data we’ve used so far for modeling transistor noise because it’s too smart to. If *ARRL Radio Designer* senses that two-port data represents a *passive* (lossy) network, it can determine that network’s

¹*ARRL Radio Designer* models noise on the basis of principles described in G. Vendelin, A. M. Pavio, and U. L. Rohde, *Microwave Circuit Design Using Linear and Nonlinear Techniques* (New York: John Wiley and Sons, 1990), and R. A. Pucel and U. L. Rohde, “An Accurate Expression for the Noise Resistance R_n of a Bipolar Transistor for Use with the Hawkins Noise Model,” *IEEE Microwave and Guided-Wave Letters*, Vol 3, No. 2, Feb 1993, pp 35-37.

Table 2

Databank Data with Noise Parameters

0.1GHZ	.74	-40	25.80	156	.017	81	.93	-14
0.5GHZ	.61	-126	12.46	108	.035	44	.57	-31
1.0GHZ	.57	-161	6.80	87	.040	38	.46	-33
1.5GHZ	.57	-180	4.67	75	.052	47	.43	-34
2.0GHZ	.58	166	3.55	64	.060	54	.41	-38
2.5GHZ	.59	160	2.92	59	.068	58	.40	-39
3.0GHZ	.61	150	2.42	50	.080	63	.39	-46
3.5GHZ	.62	142	2.09	41	.091	61	.41	-54
4.0GHZ	.62	134	1.84	32	.106	59	.42	-62
NOI RN								
0.1GHZ	1.3	.12	5	.17				
0.5GHZ	1.3	.10	25	.17				
1.0GHZ	1.4	.06	50	.16				
2.0GHZ	1.7	.25	172	.16				
4.0GHZ	3.0	.48	-131	.24				
END								

The device is an Avantek AT-41485N operating with a V_{CE} of 8 V and an I_C of 10 mA.

noise contribution (on the basis of thermal modeling) usefully enough, but if the data represents a network with gain—an *active* network—*ARD* intentionally *does not* attempt to model that network’s noise. What may be inside an *active* black box is just too unpredictable for simple thermal modeling to work.

Many RF transistor manufacturers, however, especially those who produce parts designed for frequencies at which every fraction of a decibel of noise figure counts, also supply databank data that characterizes their devices’ noise performance in addition to just listing the devices’ S parameters by frequency. Table 2 shows two-port data of this type for an Avantek AT-41485N bipolar junction transistor operating with a V_{CE} of 8 V and an I_C of 10 mA.

The noise-parameter part of the data—

NOI RN				
0.1GHZ	1.3	.12	5	.17
0.5GHZ	1.3	.10	25	.17
1.0GHZ	1.4	.06	50	.16
2.0GHZ	1.7	.25	172	.16
4.0GHZ	3.0	.48	-131	.24

—lists, by frequency, the AT-41485N’s minimum noise figure in dB (F_{min}), the magnitude and phase of the AT-41485N’s optimum noise input reflection coefficient (G_{opt}), and the AT-41485N’s equivalent noise resistance (R_n). (For details on the form of this data, see page 17-12 of *The ARRL Radio Designer Manual*.) A simple netlist (Table 3) can show us how this data plays; all it does is characterize an *ARD* two-port element (labeled AVANTBIP) with the AT-41485N data.

Figure 1 graphs the results in terms of forward transmission gain (MS_{21}) and noise figure (NF)—and it’s obvious that something’s seriously wrong. The noise figure trace oscillates between wacky, numerical-noise-ridden values for half of the graph and then, just as unrealistically, plummets to zero for the remainder. MS_{21} looks believable until the noise figure drops to zero and stays there, and then takes off for the stratosphere.

Figure 2 shows how our AT-41485N model is *supposed* to work. The difference is that *ARRL Radio Designer 1.0*, the noise modeling ability of which is *buggy* when two-port data with noise parameters comes into play, produced Figure 1, and *ARRL Radio Designer 1.5*, which we’re already shipping in response to *ARRL Radio Designer 1.0* orders, produced Figure 2!

ARRL Radio Designer 1.5 and How to Get It

We’re calling *ARD 1.5* a *maintenance release* because it’s mainly intended to fix this darned “two-port data with noise parameters” bug.² It does include a few enhancements, though, such as 3-D dialogs and controls (Figure 3).

Table 3

Testing ARD 1.0's Performance with Two-Port Data that Includes Noise Parameters

```

BLK
  TWO 1 2 0 AVANTBIP
  NOISETST:2POR 1 2
END
FREQ
STEP 0.1GHZ 4.0GHZ 0.1GHZ
END
DATA
  AVANTBIP:S
  0.1GHZ .74 -40 25.80 156 .017 81 .93 -14
  0.5GHZ .61 -126 12.46 108 .035 44 .57 -31
  1.0GHZ .57 -161 6.80 87 .040 38 .46 -33
  1.5GHZ .57 -180 4.67 75 .052 47 .43 -34
  2.0GHZ .58 166 3.55 64 .060 54 .41 -38
  2.5GHZ .59 160 2.92 59 .068 58 .40 -39
  3.0GHZ .61 150 2.42 50 .080 63 .39 -46
  3.5GHZ .62 142 2.09 41 .091 61 .41 -54
  4.0GHZ .62 134 1.84 32 .106 59 .42 -62
  NOI RN
  0.1GHZ 1.3 .12 5 .17
  0.5GHZ 1.3 .10 25 .17
  1.0GHZ 1.4 .06 50 .16
  2.0GHZ 1.7 .25 172 .16
  4.0GHZ 3.0 .48 -131 .24
END

```

How Much Does it Cost?

The ARRL Radio Designer 1.5 upgrade will be available September 1. It's free if you bought your copy of ARRL Radio Designer 1.0 on or after August 1, 1995, and \$10 if you bought your copy of ARRL Radio Designer 1.0 on or before July 31, 1995.

How Do I Get It?

Contact ARRL Publication Sales at ARRL HQ (voice 203-594-0200 and fax 203-594-0303, or e-mail ltardette@arrrl.org) for details. If you didn't purchase ARD directly from ARRL, please be sure to mention your point of purchase.

²As readers who run the Table 3 netlist in one of ARRL Radio Designer's superset programs—*Super-Compact*, *Microwave Harmonica*, and *SCOPE*—will discover, only ARRL Radio Designer 1.0 suffers from this active-TWO noise-modeling bug. ARD's professional superset programs model active-TWO noise just fine!

New Products

PAR ELECTRONICS 2-METER INTERMOD FILTERS

◊ Take a drive through any urban setting and you'll quickly realize that *intermod* is a serious problem for 2-meter operators. Emissions from high-power commercial transmitters (paging systems, in particular) mix with other signals to create interference that can effectively block the 2-meter signals you want to hear.

PAR Electronics, a long-time manufacturer of filters for the CATV and MATV industry, now offers a line of filters designed to tackle this persistent problem. Their VHF DN152 filter is designed to eliminate pager signals from 152 to 153 MHz. Insertion loss is virtually nil and the filter causes no attenuation of 70-cm signals. This makes it ideal for use with 2-meter/70-cm dual-band transceivers. A smaller model (rated at 20 W rather than 50 W) features male and female BNC connectors, allowing it to be used easily with H-Ts. Both models include rugged brass enclosures and are priced at \$62 and \$68, respectively.

PAR also offers a filter designed for 2-meter packet and satellite users. The VHF DN152Q can eliminate offending signals that appear as close as 700 kHz from the desired passband.

For more information, contact: PAR Electronics, 6869 Bayshore Dr, Lantana, FL 33462, tel 407-586-8278, fax 407-582-1234. **QST**

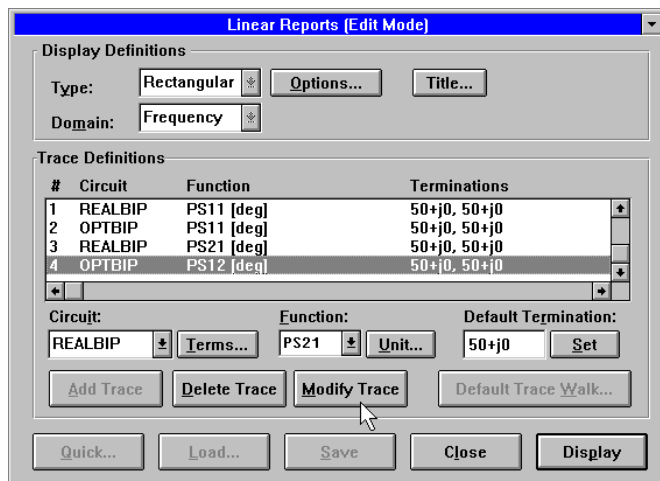


Figure 3—In addition to fixing the active-TWO noise modeling bug, ARD 1.5 also introduces three-dimensional controls and dialog boxes.

Free Electronic Goodies

The Table 2 netlist is available (as the file EXRF9509.CKT) from ARRL HQ's Internet info server (send e-mail to info@arrrl.org with a message text consisting of just the word *HELP*). A ZIP file containing EXRF9509.CKT and its accompanying ARRL Radio Designer report file (EXRF9509.RP2) is available from the ARRL HQ BBS (203-594-0306) as EXRF9509.ZIP, and via FTP (as [exrf9509.zip](ftp://arrrl.org/pub/ard)) from <ftp://arrrl.org/pub/ard> and <ftp://oak.oakland.edu/pub/hamradio/arrrl/infoserver/ard>. And ARRL Radio Designer's World Wide Web home page resides at the URL <http://arrrl.org/ard/ardpage.html>.

ARRLCAD, an e-mail reflector (mailing list) through which you can now share your questions, answers, ideas and views about ARRL Radio Designer and other Amateur-Radio-related circuit and antenna design and simulation tools with other users, is up, running and growing. To subscribe to ARRLCAD, send an e-mail message to listproc@tapr.org

with text that reads

subscribe arrlcad FirstName LastName

in which *FirstName* and *LastName* mean exactly that. (I subscribed with the message `subscribe arrlcad david newkirk`, for example.) The reflector software will confirm your subscription with a informational welcome message. Subscribing to and participating in ARRLCAD cost *nothing at all*. **QST**

ARRL Radio Designer as a Learning (and Just Plain Snooping) Tool

After all the heavy-duty transistor modeling stuff I've written for the past several columns, I want to get back to pointing out that the equally big news about *ARRL Radio Designer* is that you don't have to be a Junior Radio Engineer to get a lot out of using it. It's also *fun*, especially if you like just knowing technoidy things about radio. If you can get reasonably conversant with basic radio-design concepts and language—to the point where you know right off the top of your head what resistance and reactance are, say, and the units for and conventions of describing them, and how the reactance of certain types of radio parts changes with frequency—*ARRL Radio Designer* can be a great intuition accelerator. It can make much of the abstract technical stuff you learned in studying for your ham license come alive at your fingertips.

Reactance, impedance and resonance effects are particularly worth exploring with *ARRL Radio Designer* because they're close at hand whenever we do radio. Reactance, remember, is a component's *ac* resistance—and radio is high-frequency ac. A pure, ideal 50- Ω resistance of “looks like” (as we traditionally say) 50 Ω at any frequency. (Any *real* 50- Ω resistor, one you can actually get your hands on, will start looking less and less like a pure resistance as we move higher and higher in frequency because it exhibits some inductance and capacitance. An *ideal* resistor won't—but you also can't buy one.) On the other hand, a component with a *reactance* of 50 Ω can have that particular reactance at only one frequency; at other frequencies, its ac resis-

tance will be different. The reactance of a coil—an *inductor*—increases as frequency rises, and vice versa; the reactance of a capacitor decreases as frequency rises, as shown in Figure 1. By tradition—some prefer to dignify this a bit more and say “by convention”—we say that inductive reactance is positive and capacitive reactance is negative.

Figure 1 also shows the reactance versus frequency of a coil and capacitor connected in parallel. At the frequency at which the absolute values—the numeric parts, regardless of sign—of their reactances are *exactly equal*, they *resonate*. At this frequency—just above 5.5 MHz for the 9.1- μ H and 91-pF parts used in the Figure 1 simulation—they exhibit a high resistance, and no reactance at all. Immediately above and below resonance, their reactance shifts wildly. Above resonance, they act as a capacitor; below resonance, as an inductor. This may be old hat if you've long had your radio fundamentals well in hand, but it's fascinating—and part of the soul of radio—nonetheless. Figure 2 expands on this by showing the resonant circuit's resistance *and* reactance. Yes, resonance is a strange and wonderful thing. Just go and watch that old film clip of the wind-excited Tacoma Narrows Bridge one more time!

Viewing the phenomenon of resonance with unjailed eyes can lead to some pretty interesting circuit tricks. For instance, the fact that a paralleled coil and capacitor can act as a coil (below resonance) and a capacitor (above resonance) with reactance-versus-frequency characteristics that significantly differ from an actual coil or capacitor acting all by its lonesome—compare the slopes of the three traces in Figure 1!—can be quite handy if carefully applied. Charles J. Michaels, W7XC, used this characteristic in his April 1992 Hints and Kinks piece, “A Load-Tracking L Network” (Figure 3).

Figure 4 shows a circuit, described by Warren Bruene, now W5OLY, in Chapter 18 of Keith Henney's *Radio Engineering Handbook*, fifth edition, that takes advantage of this effect in another way. (Sure, the original circuit used tubes, but I modified the drawing so today's solid-state purists won't glaze over in disdain and miss an interesting idea.) In parallel-resonating to peak Q1's output at 7.1 MHz, L1 and C1 also *series*-resonate with C2 to shunt 3.55-MHz energy to common. In other words, L1C1

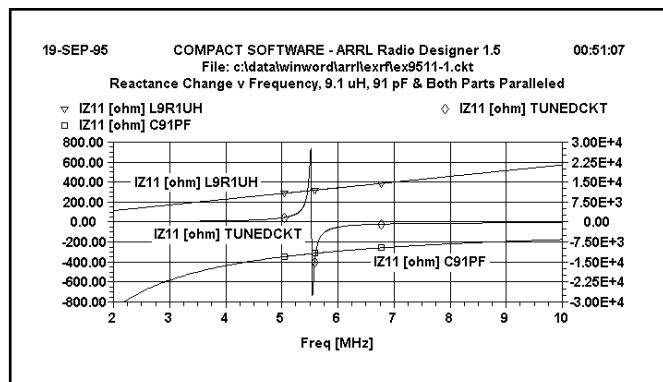


Figure 1—Maybe *you're* so well-versed in radio fundamentals that this *ARRL Radio Designer 1.5* graph of reactance versus frequency for a coil (9.1 μ H, as shown by the IZ11 [ohm] L9R1UH trace) and capacitor (91 pF, as shown by the IZ11 [ohm] C91PF trace) isn't interesting, but for many of us, me included, seeing such abstractions come to life on-screen makes ARD an untiringly powerful tool for understanding radio's basics. (This graph shows reactance as the imaginary [I] portion of the Z parameter Z_{11} for the coil and the capacitor.) Not only do we get to see—for *any* component values we care to enter, over *any* frequency range we want—how the reactance of an inductor increases with frequency and the reactance of a capacitor decreases with frequency, we also get to see the near-magical phenomenon of *resonance*: what happens when the absolute values of a parallel (or series) coil's and capacitor's reactance are exactly equal. For a 9.1- μ H coil and a 91-pF capacitor, that condition occurs at just over 5.5 MHz. At resonance, the reactance of the paralleled parts is *zero*, and together they act as a pure, high resistance.

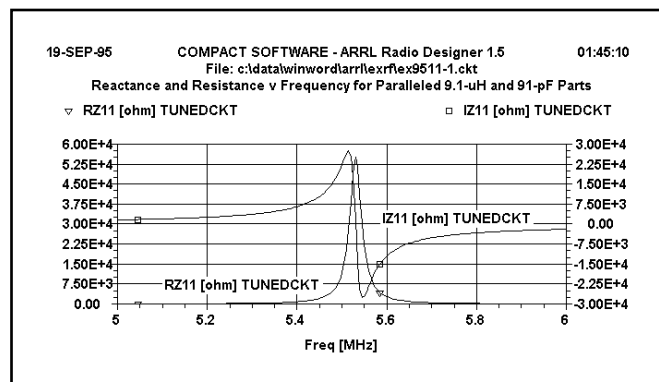


Figure 2—As their summed reactances do a wild dance around and through resonance, a paralleled inductor and capacitor exhibit a resistance that rises to a high peak right at resonance.

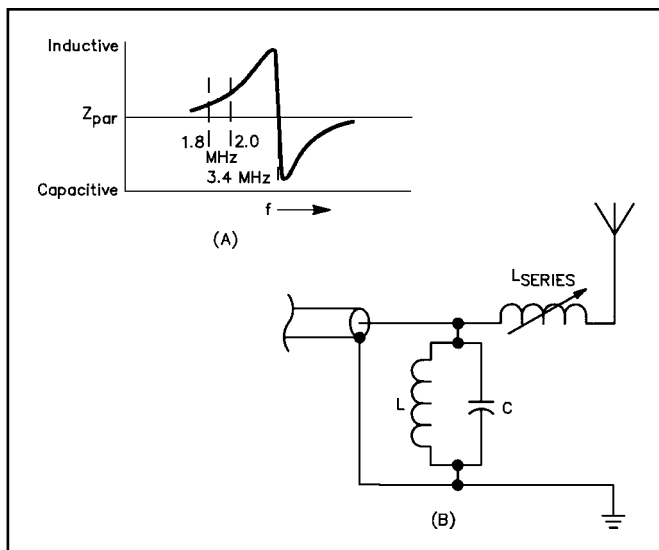


Figure 3—In a region far below resonance, where it's still inductive, the impedance-versus-frequency curve (A) for a parallel LC circuit approximates just the characteristic W7XC needed to track his antenna's impedance-at-resonance curve. Charlie Michaels configured the resulting network (B) by adjusting L and C for a match that provides an SWR of 1.4 or less from 1.8 to 2 MHz with L_{SERIES} adjusted for minimum SWR (1:1 at 1.94 MHz). In his case, L and C resonate near 3.4 MHz. (from Charlie's "A Load-Tracking L Network," April 1992 Hints and Kinks)

and C2 form a series-resonant *trap*.

Figure 5 shows how the Figure 4 circuit performs when we look across it as a two-port network and tell *ARD* to evaluate its forward transmission gain in decibels. (The logarithmic nature of the decibel scale lets us see more wide-range detail than the linear scales we'd have to use in evaluating the network's reactance, resistance or impedance.) We see the dip (at 3.55 MHz or so) and peak (at 7.1 MHz) we expect. We also see that the 1-mH RF choke gives rise to a strong resonance just above 400 kHz—something we may not want if Q1 or the subsequent stage has enough gain at 400 kHz to oscillate or regenerate. And we get an idea (from the MS21 [dB] PARNOTRP trace) of the improvement in subharmonic (fundamental)-signal rejection to be gained by this technique, compared to just letting L1 and C2 do the job on their own.

Using ARD to Snoop in Commercial Gear

Anyone who's paid much attention to the technical pages of *QST* for at least the past half-decade or so will probably roll their eyes when I yet again characterize myself as a receiver snob. But we spend most of our on-air time *listening* and relatively little of it transmitting—so we owe ourselves receivers we can not only stand to listen to, but *like* to listen to, for long stretches. A recent pair of *QST* articles by Jukka Vermasvuori, OH2GF—"Hot-Rod Your ICOM IC-725-Series Transceiver," in September and October 1995 *QST*—was music to my ears in this regard because Jukka went to considerable lengths to improve, among other things, the receive sound of the popular IC-725. I consider it quite instructive—not to mention great fun—to let *ARRL Radio Designer* confirm what Jukka found and fixed in the IC-725's audio chain—to prove yet again that if you can read a schematic and spend a few minutes typing simple alphanumeric gorp into *ARRL Radio Designer's* ASCII netlist editor, you can find out (or confirm) some quite interesting things about why your radio sounds the way it sounds. To that end, Figure 6 repeats OH2GF's schematic of the IC-725's audio chain, this time with node numbers added for *ARD* netlisting. Figure 7 shows *ARD's* simulation of the audio chain's frequency response before (UNMOD) and

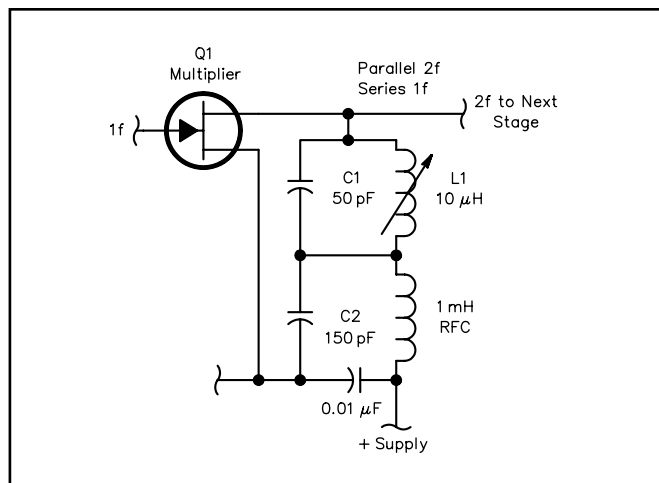


Figure 4—Just suppose you had a single-transistor frequency multiplier stage that you drove with 3.55-MHz energy to get 7.1 MHz output. If you just peaked Q1's drain with a tuned circuit (L1 and C1), you might get enough rejection of the 3.55-MHz fundamental at Q1's output, and you might not. If, however, you installed C2 to series-resonate L1C1's reactance at 3.55 MHz, you'd short-circuit much of any 3.55-MHz feedthrough to ground. The 1-mH RF choke is there to let dc flow around C2—but it has another effect, as Figure 5 shows.

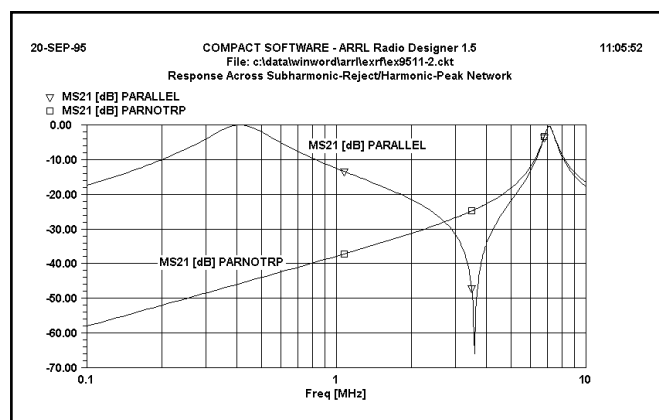
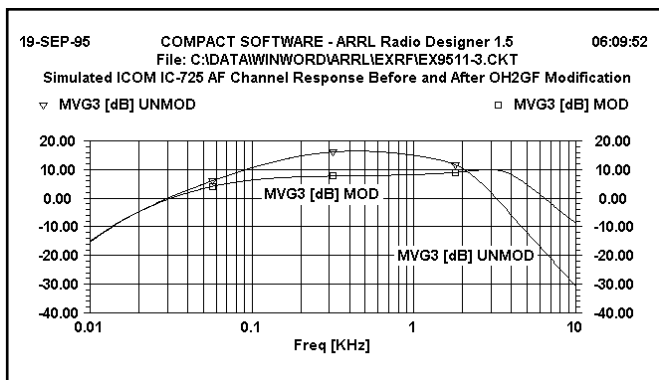
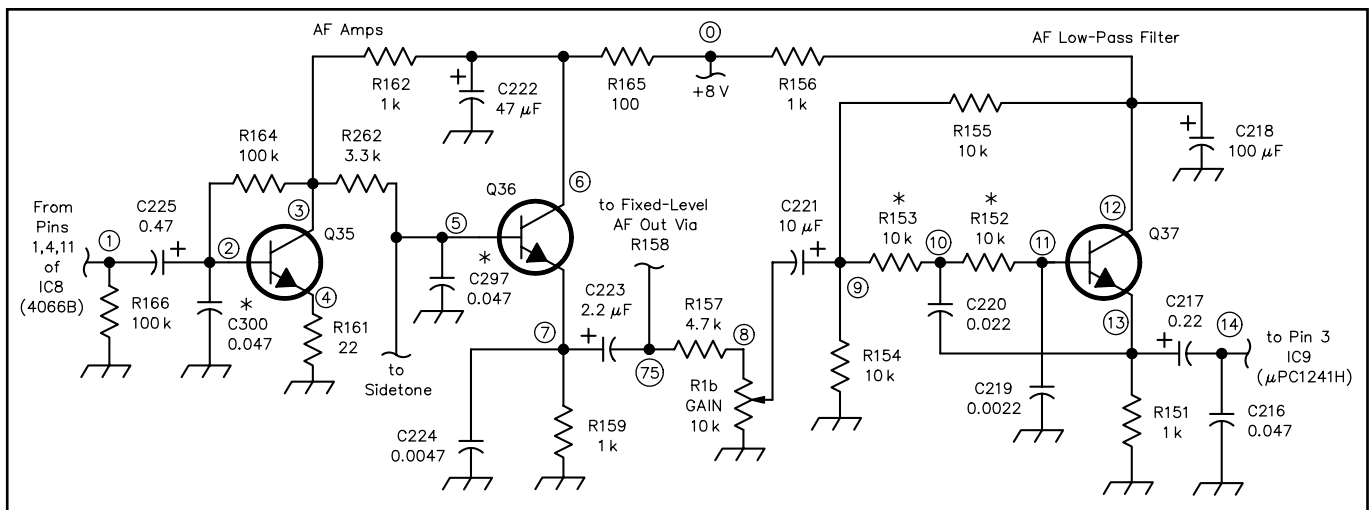


Figure 5—Well, looky here! As we expect, we see a significant peak at 7.1 MHz (where L1 and C1, Figure 4, resonate). We also see a deep dip near 3.55 MHz, where C2's reactance series-resonates with that of the L1C1 tuned circuit. The Figure 4 circuit would therefore do what we want: Peak signals at 7.1 MHz and dip signals at 3.55 MHz. But it turns out that our 1-mH RF choke isn't as benign as we thought: It parallel-resonates (through the 0.01-μF bypass capacitor) with C2 to make a strong peak near 400 kHz. Old-timers who've toiled long at the workbench trying to rid a vacuum-tube transmitter of *low-frequency parasitics* can regale you with horror stories about the hassle such hidden low-frequency resonances can cause. The MS21 [dB] PARNOTRP curve lets us compare what the 7.1-MHz L1C1 circuit would do on its own without the extra resonances added by C2 and the RF choke.

after (MOD) OH2GF's modifications. Need I say which one I'd rather listen to? The not-so-hidden message here is that if you can simulate it, you can also safely and reversibly tweak it and improve it *long before you ever warm up your soldering iron* to go to work on the real thing.

Internet Extensions to *ARRL Radio Designer*

This is where, in previous columns, I'd cut loose with the

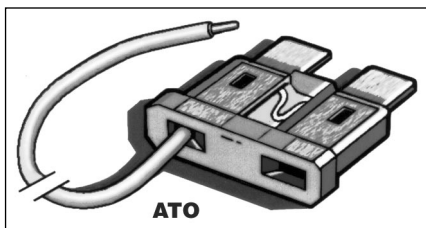


subhead **Free Electronic Goodies**—but talk about old hat! By now you've probably heard about ARRLWeb, the League's new HQ-based World Wide Web site at the URI <http://www.arrrl.org>. ARRLWeb extends ARRL *Radio Designer* in the sense that it includes a link to ARD's own home page (<http://arrrl.org/ard/ardpage.html>) and the ARD FTP site (<ftp://arrrl.org/pub/ard/>), from which a file containing circuit and report files for all of this month's column's simulations, *exrf9511.zip*, is freely available. The ARD home page extends what there is to know about ARRL *Radio Designer* in a number of ways—news, tips and files; a glimpse of how ARD looks when running under *Windows 95*; and how to subscribe to ARRLCAD, the mailing list devoted to ARD and other Amateur Radio CAD packages and topics. There's even a link to Compact Software (<http://www.comsoft.com/>), the developers and programmers of ARRL *Radio Designer* and its superset, *Super-Compact*. We're committed to making increasing use of the vast tool called the Internet, specifically the World Wide Web, to enhance the utility of ARRL *Radio Designer*.



New Products

PIGTAIL AUTOMOTIVE FUSES



Now it's easier than ever to install mobile radios or other electronic gear quickly and safely, thanks to the Pigtail Automotive Fuse from MCM Electronics. Pigtail Automotive Fuses come in ATO or Mini ATO fuse sizes now common in most vehicles. To wire your gear, replace the existing fuse with the comparable MCM Pigtail Fuse of the same value. Then splice the pigtail to any type inline connector.

Pigtail Automotive Fuses are available in 7.5, 10, 15 and 20-amp sizes. They're \$2.49 for a bag of five (\$20 minimum order, plus shipping and handling). For more information or a free catalog, call 513-434-0031 or toll-free, 800-543-4330, fax 513-434-6959 or write MCM Electronics, 650 Congress Park Drive, Centerville, OH 45459-4072.

MFJ ADDS GPS COMPATIBILITY TO THE MFJ-1270C TNC

▷ MFJ has added Global Positioning System (GPS) compatibility to its popular MFJ-1270C VHF/HF packet controller. GPS, when used with Automatic Packet Reporting System (APRS) software, permits fairly precise tracking of the position and movement of suitably equipped vehicles—cars, boats, planes and the like—or even individuals.

In addition to GPS compatibility, the unit offers standard VHF/HF TNC features, including personal mailbox (expandable to 512k) and WeFAX. A plug-in

modem for 2400 or 9600 baud is optional. The MFJ-1270C is fully TAPR-2 compatible.

The GPS-compatible MFJ-1270C lists for \$119.95. An optional starter pack, including interface cables, software and instructions, is \$24.95 for the DOS version or \$29.95 for the Microsoft *Windows* version. Software for Commodore 64/128 or Macintosh machines also is available. A pre-wired radio-to-TNC cable (available for many popular rigs) is \$14.95.

For more information, call MFJ Enterprises, 601-323-5869, fax 601-323-6551, or write PO Box 494, Mississippi State, MS 39762. The MFJ toll-free order line is 800-647-1800.

